14 haftalık yol haritasında 12. hafta

ALGORİTMA VE PROGRAMLAMA – I · DERS NOTU

HAFTA 12

İşaretçiler: Adreslerle Programlama ve Belleğin Denetimi

Adres ve dolaylama, kutu–ok modeli, referansla aktarım, işaretçi aritmetiği ve diziler, const, yığın ve öbek, malloc/calloc/realloc/free, sanal bellek, bellek hataları ve AddressSanitizer/Valgrind

MODÜL V · VERİ TOPLULUKLARI VE BELLEK · H10–H12

> HAFTANIN BÜYÜK SORUSU
>
> *H8'de yazdığımız takas fonksiyonu neden çalışmadı ve bir değişkenin adresini bilmek bize hangi gücü ve hangi sorumluluğu verir?*

Dr. Öğr. Üyesi Mehmet Sevri
Recep Tayyip Erdoğan Üniversitesi · Mühendislik ve Mimarlık Fakültesi · Bilgisayar Mühendisliği Bölümü
Bilgisayar Mühendisliği 1. sınıf · 2026–2027 Güz yarıyılı · Sürüm 1.0 · Eylül 2026
`Derleme ortamı: GCC (C17) · gcc -std=c17 -Wall -Wextra -pedantic`

İçindekiler

1. Haftanın Büyük Sorusu ve Öğrenme Çıktıları 3
1.1 Öğrenme çıktıları 3
2. Geçen Haftadan: Matristen Adrese 3
3. Kavramsal Anlatım 4
3.1 Adres: bellek, numaralı baytlardan oluşan bir dizidir 4
3.2 İşaretçi: adres tutan değişken; & ve * operatörleri 5
3.3 Dolaylama ile okuma ve yazma 6
3.4 NULL ve güvenli ilk değer 7
3.5 Referansla aktarım: takas, scanf ve çıktı parametreleri 7
3.6 İşaretçi aritmetiği ve dizi–işaretçi ilişkisi 8
3.7 const ve işaretçiler 10
3.8 Kaputun altında: yığın, öbek ve sanal bellek 10
3.9 Dinamik bellek: malloc, calloc, realloc, free 12
3.10 Bellek hataları, AddressSanitizer ve Valgrind 12
4. Çözümlü Örnekler: Örnek Merdiveni 13
4.1 Amiral örnek — Örnek 3 ★★: referansla takas 13
4.2 Örnek 4–6: çıktı parametreleri ve p++ 15
4.3 Örnek 8–11: iki işaretçi, const, malloc, baytlar 16
4.4 Amiral örnek — Örnek 12 ★★★: realloc ile büyüyen dizi 18
4.5 Örnek 13–14: hata avı ve fonksiyon işaretçileri 20
5. Hata Avcısı 21
6. Mühendis Gözüyle: Sahiplik, Araçlar ve Bellek Güvenliği 22
7. Mini-OBS v0.12: Dinamik Diziler ve İşaretçi Parametreler 23
8. Alıştırmalar ve Atölye 25
9. Kendini Değerlendirme 28
10. AP-2 Köprüsü ve Merak Kutusu 28
11. Özet, Kavram Haritası ve Terim Sözlüğü 29
12. Kaynaklar 30

1. Haftanın Büyük Sorusu ve Öğrenme Çıktıları

H8'de yazdığımız takas fonksiyonu neden çalışmadı ve bir değişkenin adresini bilmek bize hangi gücü ve hangi sorumluluğu verir?

H8'de iki değişkeni değiş tokuş eden bir `takas(a, b)` fonksiyonu yazdık: fonksiyonun içinde değerler değişti, ama `main`'deki değişkenler yerinde kaldı (`H08_10_takas_denemesi.c`). H9'da nedenini gördük: her çağrının kendi **yığın çerçevesi** ve kendi kopyaları vardır; çerçeve dönüşte yok olur. Bu hafta eksik parçayı ekliyoruz: bir değişkenin **adresini** taşıyan değişken, yani **işaretçi** (pointer). Adresi bilen bir fonksiyon başka bir çerçevedeki kutuya uzanabilir, çalışma zamanında ne kadar bellek gerekiyorsa o kadarını isteyebilir: **güç** buradadır. Yanlış ya da süresi dolmuş bir adresi kullanan program ise kendi belleğini sessizce bozar: **sorumluluk** da buradadır.

> GERÇEK DÜNYADAN • **1965–2009: Hoare'un "milyar dolarlık hatası"**
>
> **Tony Hoare**, 2009'da QCon London konferansındaki konuşmasında, 1965'te ALGOL W dili için tasarladığı başvuru (reference) tip sistemine **null referansı** eklediğini anlattı ve bu kararı "milyar dolarlık hatam" (*my billion-dollar mistake*) diye nitelendirdi. Konuşma özetine göre amacı başvuruların bütün kullanımının derleyicinin otomatik denetimiyle mutlak güvenli olmasıydı; null referansı ise "yalnızca gerçekleştirmesi çok kolay olduğu için" (*simply because it was so easy to implement*) eklemişti. Özetteki değerlendirmesiyle bu karar sayısız hataya, güvenlik açığına ve sistem çökmesine yol açmıştı (Hoare, 2009).
>
> C'deki karşılığı **NULL işaretçidir**; onu dolaylamak tanımsız davranıştır; Linux/Windows'ta program genellikle SIGSEGV ile sonlanır (bkz. HA4). Sorun güncel: Chromium projesi, 2015'ten bu yana Stable kanalını etkileyen 912 yüksek ve kritik önem dereceli güvenlik hatasının yaklaşık %70'inin bellek güvenliği sorunu olduğunu, bunların yarısının free sonrası kullanım olduğunu bildirir (Chromium Project, t.y.); Microsoft da CVE atadığı açıkların yaklaşık %70'inin bellek güvenliği sorunu olmayı sürdürdüğünü açıklamıştır (MSRC Team, 2019).

1.1 Öğrenme çıktıları

ÖÇ 12.1 İşaretçi bildirir; adres (&) ve dolaylama (*) operatörlerini doğru kullanır ve işaretçi–değer ilişkisini kutu–ok diyagramıyla gösterir. *(Bölüm 3.1–3.3, Örnek 1–2)*

ÖÇ 12.2 Referansla aktarım (işaretçi parametreler) ile fonksiyondan birden çok sonuç döndürür. *(Bölüm 3.5, Örnek 3–5)*

ÖÇ 12.3 İşaretçi aritmetiğini ve dizi–işaretçi ilişkisini (a[i] ≡ *(a + i)) açıklar. *(Bölüm 3.6, Örnek 6–8)*

ÖÇ 12.4 NULL, başlatılmamış ve sarkan (dangling) işaretçi hatalarını tanır ve önler. *(Bölüm 3.4, 3.10, Bölüm 5, Örnek 13)*

ÖÇ 12.5 Yığın (stack) ve öbek (heap) belleğini ayırt eder; malloc/calloc/realloc/free ile dinamik dizi oluşturur ve yönetir. *(Bölüm 3.8–3.9, Örnek 10–12, Bölüm 7)*

2. Geçen Haftadan: Matristen Adrese

H11'de matrisleri **satır öncelikli** yerleşimle bellekte tek boyutlu bir şerit olarak gördük ve adresi formülle hesapladık. Bu hafta o formülü bir **değişkende** tutacağız. Önce üç geri çağırma sorusu (`kodlar/quiz/`):

> **TAHMİN ET · Geri çağırma: üç soru**
>
> - **S1 (satır öncelikli yerleşim).** `int m[3][4]`'te `m[2][1]`, `m[0][0]`'dan kaç bayt ileridedir? (`int` 4 bayt) (`H12_Q01_satir_oncelikli.c`)
> - **S2 (2B dizi parametresi).** Program ne yazdırır? Parametrede `SUTUN` neden zorunlu? (`H12_Q02_2b_parametre.c`)
> - **S3 (komşuluk ve sınır).** Bütün hücreleri 1 olan 3 × 3 ızgarada köşe, kenar ve orta hücrenin 8-komşu toplamı? (`H12_Q03_komsuluk.c`)

Yanıtlar (gerçek çıktılar). S1: "m[2][1], m[0][0]'dan 36 bayt ileride": (2 × 4 + 1) × 4 = 36 (programdaki `uintptr_t`, `<stdint.h>`'nin bir adresi taşıyabilen tamsayı tipidir; H09_K03'te de adres farkı için kullanmıştık, ayrıntısı Örnek 12'de). S2: "6 15"; derleyici `m[i][j]`'nin adresini taban + (i × SUTUN + j) × 4 ile hesapladığı için sütun sayısını bilmek zorundadır. Bu hafta göreceğiz: `int m[][SUTUN]` parametresi aslında bir **satır işaretçisidir**. S3: "köşe 3, kenar 5, orta 8": sınır denetimi ızgara dışındaki "komşuları" atlar.

Bu haftanın konusu, önceki haftalarda bilerek açık bıraktığımız beş sorunun ortak yanıtıdır (Tablo 12.1).

Tablo 12.1
Önceki haftalardan açık kalan sorular ve bu haftaki yanıtları

Hafta	Açık bırakılan soru	Bu haftaki yanıt
H3	`scanf("%d", &a)`'daki `&` neden gerekli? Bir `int`'in baytları bellekte hangi sırada?	scanf'e kutunun adresi gider, scanf o adrese yazar (3.5); baytları `unsigned char *` ile C içinden okuruz (Örnek 11)
H8	`takas(a, b)` neden çalışmadı?	Kopyalar değil adresler aktarılır: `takas(&x, &y)` (Örnek 3)
H9	Bellek haritasındaki öbek (heap) bölgesi ne işe yarar?	`malloc` ile çalışma zamanında istenen bellek; yığın ve öbek (3.8–3.9)
H10	Fonksiyon içinde `sizeof dizi` neden 8 verdi?	Dizi parametresi aslında işaretçidir (3.6)
H11	`m[i]` tek başına nedir?	i. satırın adresi; `int (*m)[SUTUN]` (3.6, Mini-OBS v0.12)

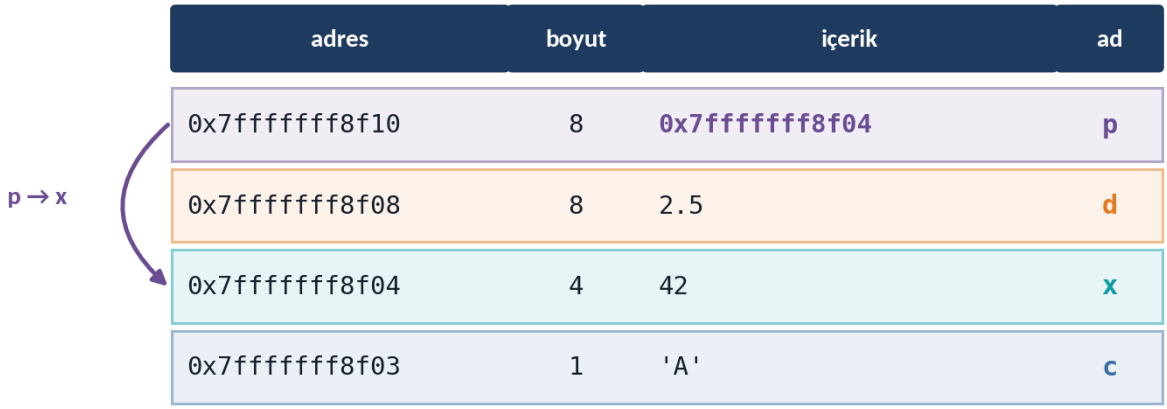
3. Kavramsal Anlatım

3.1 Adres: bellek, numaralı baytlardan oluşan bir dizidir

Belleği, her biri bir bayt tutan ve sıfırdan başlayarak numaralandırılmış kutulardan oluşan çok uzun bir **dizi** gibi düşünelim. Bir baytın **adresi**, bu dizideki **indisidir**. Birden çok baytlık bir değişken (4 baytlık `int`, 8 baytlık `double`) art arda gelen baytları kaplar; değişkenin adresi, ilk baytının adresidir. H3'te `&x` ile adresleri `%p` biçimiyle yazdırmıştık; şimdi `H12_K02_adres_tablosu.c` dört değişkenin adresini, boyutunu ve içeriğini yazdırıyor (Şekil 12.1; sunumda ve web sayfasında aynı baytlar tek tek de gösterilir).

Adresler her çalıştırmada değişir: iki çalıştırmada `x` önce `0x7fff1fff87e4`, sonra `0x7ffff072a254` adresindeydi. İşletim sistemi bölgelerin yerini her seferinde rastgele kaydırır (**ASLR**, adres alanı düzeni rastgeleleştirmesi; saldırganın adres tahminini zorlaştırır). `setarch x86_64 -R` bunu tek çalıştırma için kapatır; şekillerde kararlı adresler için bu çalıştırmayı kullanıyoruz. Farklar ise değişmez (`p`, `x`'ten hep 12 bayt yukarıda); değişkenlerin yığındaki sırasını derleyici seçer.

Şekil 12.1
Adres tablosu (adres | boyut | içerik | ad): p'nin İÇERİĞİ, x'in ADRESİDİR



Yüksek adres üstte (H12_K02, setarch -R). p'nin İÇERİĞİ = x'in ADRESİ: kutu–ok modelindeki ok.

3.2 İşaretçi: adres tutan değişken; & ve * operatörleri

İşaretçi (pointer), değeri bir **adres** olan değişkendir. Bildirimde tipin yanına `*` konur: `int *p;` "p, bir int'i gösteren işaretçidir" diye okunur. İki operatör işaretçinin iki yönüdür: **adres operatörü** `&x`, x'in adresini verir; **dolaylama** (dereference, içerik alma) operatörü `*p`, "p'nin gösterdiği yerdeki nesne" demektir (Kernighan & Ritchie, 1988, Bölüm 5; Deitel & Deitel, 2022, Bölüm 7). Bu hafta boyunca kullanacağımız **kutu–ok modeli**nde işaretçi bir kutudur; içinden, gösterdiği kutuya bir **ok** çıkar (Şekil 12.2). Örnek 1:

H12_01_adres_deger.c · satır 11–25

```c
#include <stdio.h>

int main(void)
{
    int x = 42;
    int *p = &x;                        /* p, x'in adresini tutar */

    printf("x  (değer)              : %d\n", x);
    printf("&x (x'in adresi)        : %p\n", (void *)&x);
    printf("p  (içindeki adres)     : %p\n", (void *)p);
    printf("*p (gösterdiği değer)   : %d\n", *p);
    printf("&p (p'nin kendi adresi) : %p\n", (void *)&p);
    printf("sizeof x = %zu bayt, sizeof p = %zu bayt\n", sizeof x, sizeof p);
    return 0;
}
```

Şekil 12.2
*Kutu–ok modeli (Örnek 1, ASLR kapalı çalıştırma): p'nin içindeki değer x'in adresidir; *p okun ucundaki kutudur*




```
int *p = &x;   →  p'nin içine x'in ADRESİ yazılır (8 bayt)
*p             →  okun ucundaki kutu: x'in kendisi (42)
```

Üç gözlem (ASLR kapalı gerçek çalıştırma, Şekil 12.2): `p` ile `&x` **aynı** sayıdır (`0x7fffffff8f1c`); `*p` ile `x` **aynı** kutudur (42); `p`'nin de kendi adresi vardır (`&p`): işaretçi de bellekte yaşayan bir değişkendir (`0x7fffffff8f20`). (`%p` bir `void *` beklediği için `(void *)` dönüşümü yazılır.)

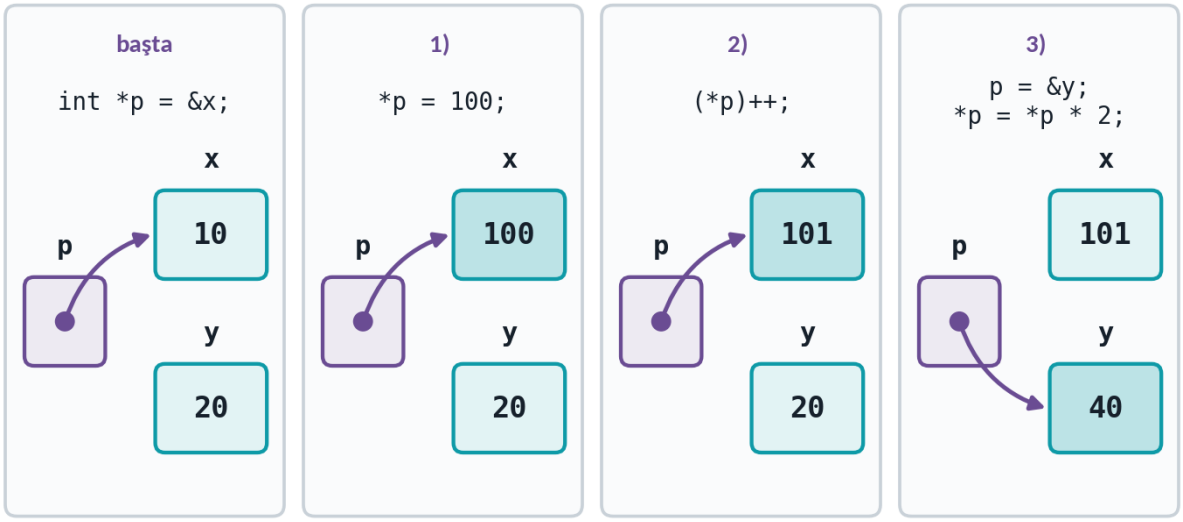
> KAPUTUN ALTINDA · **64 bitlik sistemde işaretçi 8 bayttır**
>
> Bir işaretçi, gösterdiği tip ne olursa olsun yalnızca bir **adres** taşır; x86-64 Linux'ta (LP64) adres 64 bittir: `H12_K01_sizeof_isaretci.c` "sizeof(int) = 4 sizeof(int *) = 8", "sizeof(double) = 8 sizeof(double *) = 8" yazar. 64 bit Windows'ta da işaretçiler 8 bayttır (orada `long` 4 bayttır, LLP64). Bugünkü x86-64 işlemciler sanal adresin 48 bitini (beş düzeyli sayfa tablosu destekleyenlerde 57 bitini) kullanır; işaretçi yine 8 bayt yer kaplar (Bryant & O'Hallaron, 2016, Bölüm 9). Tip ise iki şeyi belirler: `*p`'nin kaç bayt okuyacağını (`int *` 4, `double *` 8) ve `p + 1`'in kaç bayt ilerleyeceğini (3.6).

3.3 Dolaylama ile okuma ve yazma

`*p` yalnızca okumak için değil, **yazmak** için de kullanılır: atamanın sol yanındaki `*p`, "p'nin gösterdiği kutuya yaz" demektir. Aynı işaretçi başka bir kutuya yöneltilebilir. Örnek 2 (`H12_02_dolaylama_yazma.c`) bunu dört satırda yapar; her satırdan sonra diyagramı güncelleyin (Şekil 12.3, her panelin üstünde o adımın kodu):

Bir sembol, üç anlam. `*` iki işlenenle **çarpma**, bildirimde **tipin parçası** (`int *p`), tek işlenenle **dolaylamadır**; `&` iki işlenenle bit düzeyinde VE (H4), tek işlenenle **adres** operatörüdür. **Öncelik tuzağı:** sonek `++`, `*`'dan önce bağlanır: `*p++` p'yi ilerletir (Örnek 6), x'i artırmak için `(*p)++` yazılır. **Bildirim tuzağı:** `*` tipe değil **ada** bağlanır; `int *p, q;` satırında q düz bir `int`'tir (HA2). **Değiştir (Örnek 1–2).** Örnek 1'de `printf("%d", p)` yazın: derleyici ne der? Örnek 2'de `(*p)++` yerine `*p++` yazın: x değişir mi?

Şekil 12.3
Örnek 2'nin kutu–ok izlemesi (değerler gerçek çıktıdan): değişen kutu koyu, ok her adımda p'nin gösterdiği kutuya gider



3.4 NULL ve güvenli ilk değer

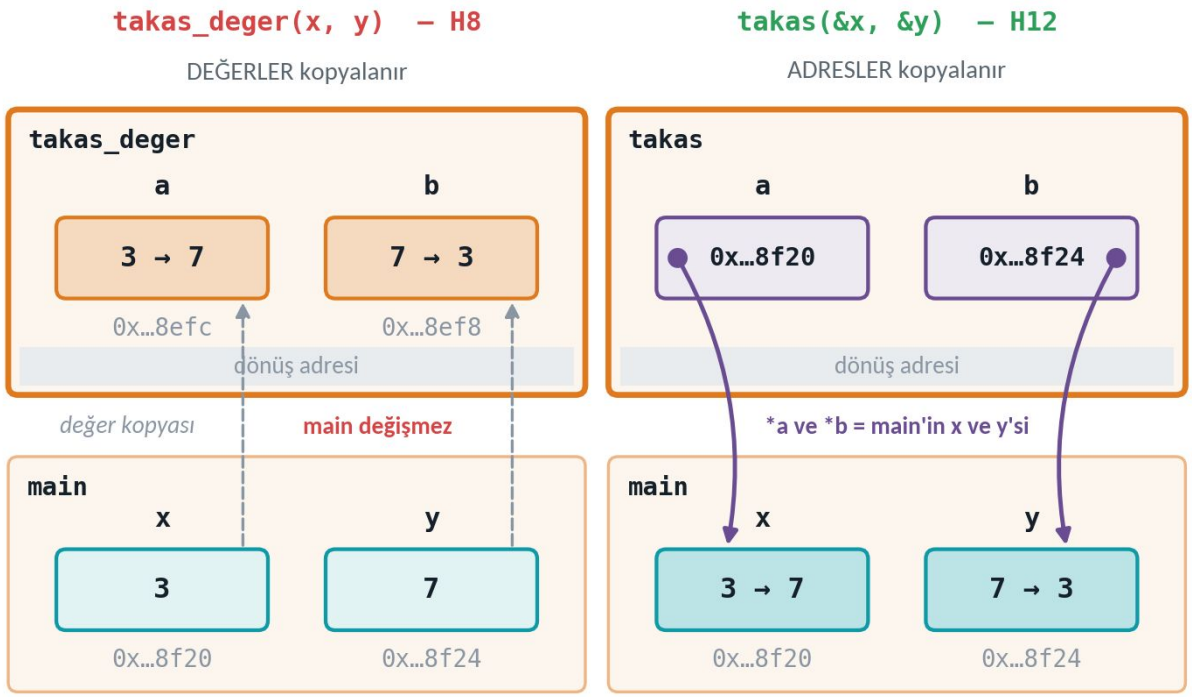
İlk değer verilmemiş yerel bir işaretçinin içinde o yığın hücresinde daha önce ne kaldıysa o vardır; ona yazmak tanımsız davranıştır ve en kötü hâlinde program **çökmeden** başka bir değişkeni bozar (HA3'te tam olarak böyle oldu). Çözüm, "hiçbir nesneyi göstermiyor" anlamındaki **NULL** değeridir (`stddef.h`, `stdio.h`, `stdlib.h`). Üç alışkanlık: (1) işaretçiye ilk değer olarak geçerli bir adres ya da NULL verin; (2) "bulunamadı" gibi durumları NULL ile bildirin ve dolaylamadan **önce** denetleyin; (3) `free`'den sonra işaretçiyi NULL yapın. C standardına göre `free(NULL)` hiçbir şey yapmaz (ISO/IEC, 2018, 7.22.3.3); bu yüzden NULL'lanmış bir işaretçiyi yanlışlıkla yeniden bırakmak zararsızdır.

`H12_K07_null_guvenli.c`'deki `bul(a, n, aranan)` aranan değeri bulursa elemanın **adresini** döndürür; çağıran bu adres üzerinden diziyi değiştirebilir (`*p = 80` → `a[1] = 80`). Bulamazsa NULL döner; çağıran `if (p != NULL)` ile dolaylamadan **önce** denetler (gerçek çıktı "a[1] = 80", "99 yok (NULL döndü)", "free(NULL) sorunsuz."). HA4'te aynı fonksiyonun dönüşü denetlenmeden dolaylanır: Hoare'un hatasının en kısa C örneği.

3.5 Referansla aktarım: takas, scanf ve çıktı parametreleri

C'de bütün argümanlar **değerle** aktarılır (H8). Bir adres verdiğimizde de adresin kopyası gider; ama kopya **aynı kutuyu** gösterir ve fonksiyon `*a` üzerinden yazınca çağıranın değişkeni değişir. Bu desene **referansla aktarım** (pass by reference) denir; C'de işaretçiyle yapılır (Şekil 12.4, Örnek 3).

Şekil 12.4
Değerle (solda) ve referansla (sağda) takas: yığın çerçeveleri ve gerçek adresler (Örnek 3, ASLR kapalı). Etkin çerçeve koyu kenarlı



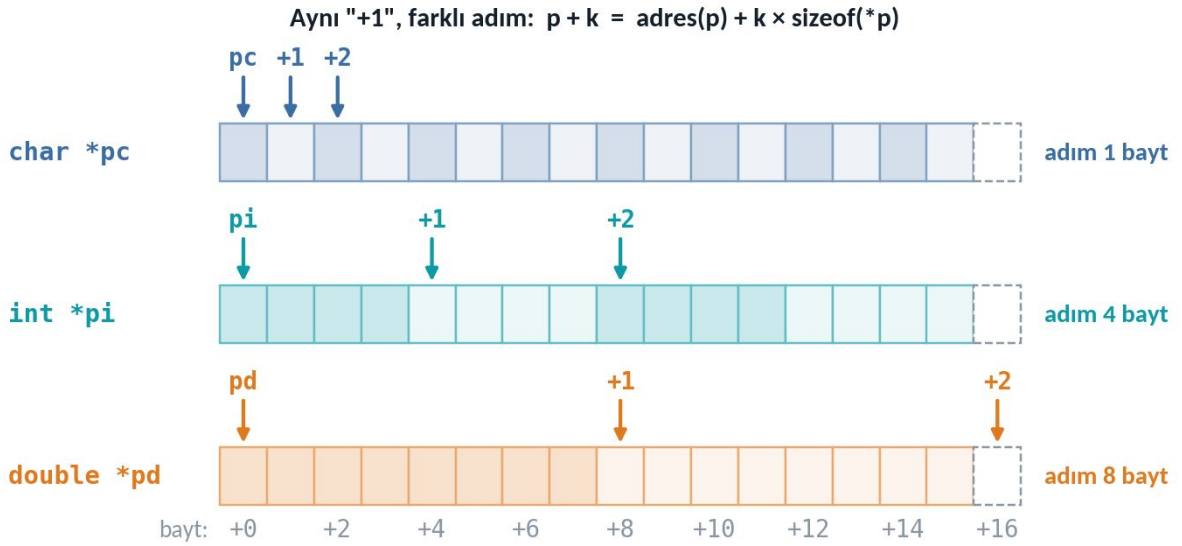
scanf'teki &'ın anlamı (H3'ün yanıtı). `scanf` okuduğu değeri bir yere **yazmalıdır**; `scanf("%d", a)` yazsaydık ona a'nın (belirsiz) değeri giderdi. `&a` ile adresi veririz, scanf oraya yazar. Elimizde zaten bir işaretçi varsa `&` yazılmaz: `H12_K04_scanf_adres.c`'de `int *p = &x;` ile `scanf("%d", p)` de x'e yazar (gerçek çıktı "x = 9").

Aynı desenle bir fonksiyon **birden çok sonuç** döndürebilir: `return` ile tek değer döner; diğer sonuçlar çağıranın verdiği adreslere yazılır (**çıktı parametreleri**). `bolum_kalan(17, 5, &bolum, &kalan)` iki sonucu birden üretir; dönüş değerini de başarı bilgisi için kullanır (Örnek 4).

3.6 İşaretçi aritmetiği ve dizi–işaretçi ilişkisi

Bir işaretçiye tamsayı eklemek adrese **o kadar eleman** eklemektir: `p + k`, adrese k × sizeof(*p) bayt ekler; `char *` 1, `int *` 4, `double *` 8 bayt ilerler (Şekil 12.5). İki işaretçinin farkı (`p - a`) aradaki **eleman sayısıdır** (`ptrdiff_t`, belirteç `%td`). Aritmetik ve karşılaştırma yalnızca **aynı dizinin** içinde anlamlıdır; dizinin bir ötesini göstermek serbesttir, dolaylamak tanımsız davranıştır.

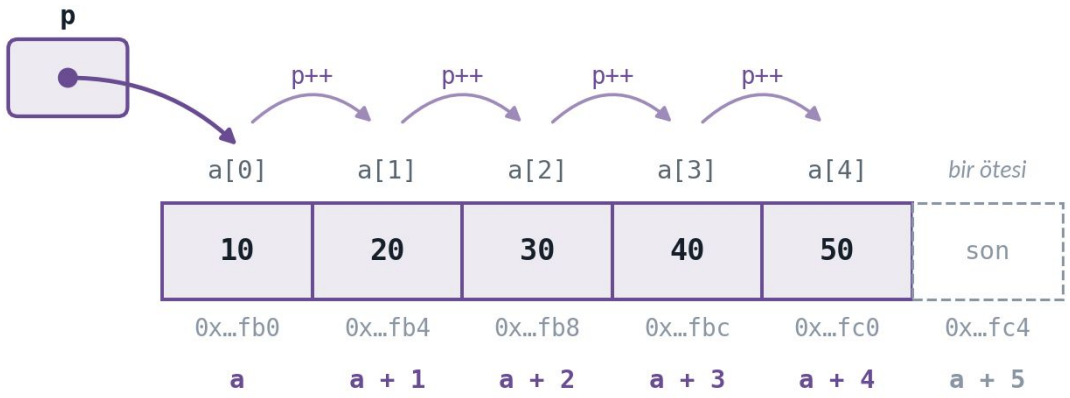
Şekil 12.5
*Aynı "+1", farklı adım: char *, int * ve double * için p, p + 1, p + 2 (adımlar H12_K03'ün gerçek ölçümü)*



Gerçek ölçüm: H12_K03_olcekleme.c (+1 adımı 1 · 4 · 8 bayt); renk tonları elemanları ayırır

Dizi adı = ilk elemanın adresi. Bir dizinin adı ifadelerin çoğunda ilk elemanın adresine **dönüşür** (a → `&a[0]`); istisnaları `sizeof a`, `&a` ve bir diziye ilk değer veren dizgi sabitidir (`char s[] = "abc";` dizgiyi diziye kopyalar; ISO/IEC, 2018, 6.3.2.1). Standart, indisleme operatörünü zaten işaretçi aritmetiğiyle **tanımlar**: `a[i]`, `*(a + i)` demektir (6.5.2.1). Toplama değişmeli olduğu için `*(i + a)`, yani `i[a]` de aynı kutudur: derlenir, ama yalnızca bir meraktır. H10'da Kaputun Altında'da "ilk bakış" dediğimiz eşdeğerlik budur (Şekil 12.6). Örnek 7 (`H12_07_dizi_isaretci_denklik.c`) dört elemanlı bir dizide `a[i]`, `*(a + i)`, `p[i]`, `*(p + i)` ve `i[a]` sütunlarını yan yana yazdırır: her satırda beş değer aynıdır; son iki satırı "a == &a[0] ? evet", "sizeof a = 16 bayt, sizeof p = 8 bayt".

Şekil 12.6
*a[i] ≡ *(a + i): dizi şeridi, adresler (Örnek 6'nın gerçek çalıştırması) ve p++ adımları; a + 5 dizinin bir ötesidir*



`a[i] ≡ *(a + i) ≡ p[i] ≡ *(p + i) ≡ i[a]`

adres farkı 4 bayt = sizeof(int) · p − a = eleman sayısı · a + 5 yalnızca karşılaştırılır, dolaylanmaz

Derleyici bu eşdeğerliği kendi iletisinde de gösterir: `const int *a` parametresine `a[i] = 0` yazan `H12_K06_const_hatalari.c`'ye GCC 13 "assignment of read-only location '*(a + (sizetype)((long unsigned int)i * 4))'" der; `a[i]` derleyicinin gözünde `*(a + i × 4)`'tür.

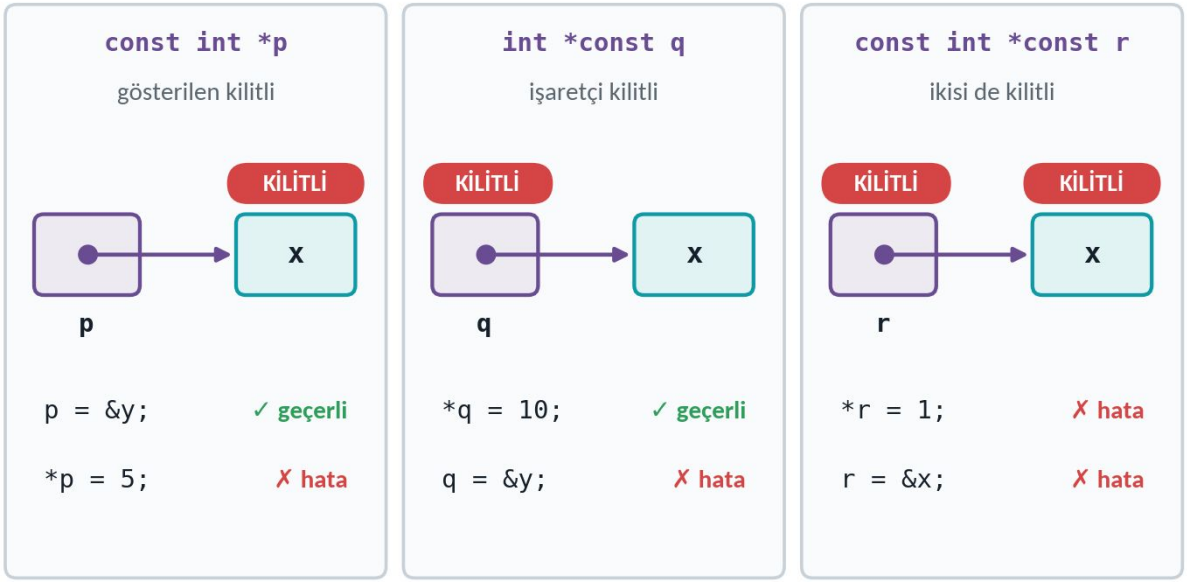
Dizi parametresi aslında işaretçidir (H10'daki sizeof tuzağı). `void f(int dizi[])` parametresi `int *dizi` demektir; çağrıda ilk elemanın **adresi** gider. `H12_K05_dizi_parametresi.c` (kasıtlı uyarı içerir) "main'de sizeof notlar = 40 (10 x 4 bayt)", "fonksiyonda sizeof dizi = 8 (int * boyutu)" yazar ve GCC uyarır: "'sizeof' on array function parameter 'dizi' will return size of 'int *' [-Wsizeof-array-argument]".

Boyutun ayrı parametreyle verilmesinin (H10) nedeni budur. H11'in `int m[][SUTUN]` parametresi de aslında `int (*m)[SUTUN]`'dur: **SUTUN elemanlık bir satırı gösteren işaretçi**; `m + 1` bir sonraki satıra (SUTUN × 4 bayt) gider, `m[i]` i. satırın adresidir. Mini-OBS v0.12 notları öbekte bu tiple tutar (Bölüm 7).

3.7 const ve işaretçiler

Bir işaretçide **gösterilen nesne** ya da **işaretçinin kendisi** kilitlenebilir; bildirimi **sağdan sola** okuyun: `const int *p` (p üzerinden yazılamaz, p yön değiştirebilir), `int *const q` (q hep aynı kutuyu gösterir, kutuya yazılabilir), `const int *const r` (ikisi de kilitli; Şekil 12.7, Örnek 9). Parametredeki `const int *a` çağırana bir **sözdür** ("okurum, değiştirmem"); derleyici bunu denetler: yasak satırlar "assignment of read-only location" ya da "assignment of read-only variable" hatasıyla reddedilir (şekildeki iletiler `H12_K06_const_hatalari.c`'nin gerçek derlemesindendir).

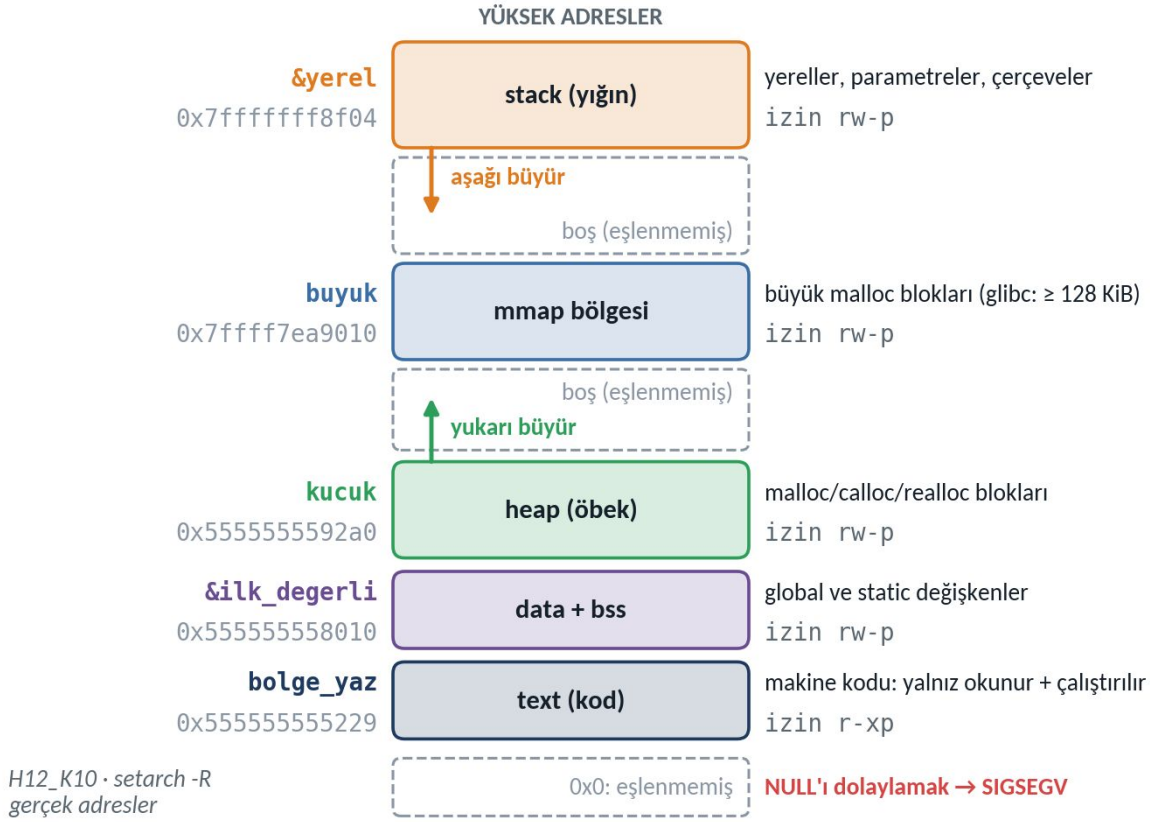
Şekil 12.7
const'un yeri neyin kilitlendiğini belirler; ✗ satırları derlenmez (H12_K06'nın gerçek iletileri)



3.8 Kaputun altında: yığın, öbek ve sanal bellek

H9'da süreç bellek haritasını çizmiş, **öbeği** (heap) bir `malloc` önizlemesiyle göstermiş, ayrıntısını H12'ye bırakmıştık. `H12_K10_bellek_haritasi.c` her bölgeden bir nesnenin adresini yazdırır ve Linux'un o süreç için tuttuğu haritada (`/proc/self/maps`) hangi bölgeye ve hangi **izinlere** (r okuma, w yazma, x çalıştırma) düştüğünü gösterir; ASLR kapalı gerçek çıktı Şekil 12.8'ye işlenmiştir (tamamı `ciktilar/` altında).

Şekil 12.8
Tamamlanan süreç bellek haritası: yığın aşağı, öbek yukarı büyür; büyük bloklar ayrı bir mmap bölgesinden gelir. Adresler ve izinler H12_K10'dan



Kod bölgesi **r-x**'tir (yazılamaz); veri, öbek ve yığın **rw-**'dir. 64 baytlık blok `[heap]` bölgesinden, 1 MiB'lık blok ayrı bir **mmap** bölgesinden geldi (glibc büyük istekleri, varsayılan eşik 128 KiB, doğrudan işletim sisteminden ister); bölge sınırları 0x1000'in katıdır: bellek **4 KiB'lık sayfalar** hâlinde verilir. **Yığın** yerel değişkenleri tutar, ömrü otomatiktir, boyutu küçük ve sınırlıdır (Linux'ta tipik 8 MiB, H9); **öbek** çalışma zamanında istenen blokları tutar, ömrünü programcı yönetir (`free` çağrılana dek). Yığının tipik hataları taşma ve yerel adresin döndürülmesi; öbeğinkiler sızıntı, free sonrası kullanım, çifte free ve taşmadır.

> KAPUTUN ALTINDA · **Sanal bellek, MMU ve "Segmentation fault"**
>
> Programın gördüğü adresler **sanal adreslerdir**. İşlemcideki bellek yönetim birimi (**MMU**), her erişimde sanal adresi işletim sisteminin tuttuğu **sayfa tablosu** yardımıyla fiziksel RAM'deki bir çerçeveye çevirir ve sayfanın izinlerini denetler (Bryant & O'Hallaron, 2016, Bölüm 9). Her sürecin kendi sayfa tablosu vardır: iki sürecin aynı sanal adresi farklı fiziksel çerçevelere düşer; bir süreç ötekinin belleğini göremez (sunumdaki ve web sayfasındaki sanal bellek şeması).
>
> Eşlenmemiş bir sayfaya (ör. NULL'ın bulunduğu sıfır sayfası) ya da izni olmayan bir erişime (ör. salt okunur sayfaya yazma) MMU bir istisna üretir; işletim sistemi süreci **SIGSEGV** sinyaliyle sonlandırır, kabuk "Segmentation fault" yazar ve çıkış durumu 128 + 11 = **139** olur. `H12_K11_koruma_ihlali.c`'nin iki seçeneği de gerçekten böyle sonlandı: NULL'dan okuma ve salt okunur bir `static const` nesneye yazma. Segmentation fault bir **koruma ihlalidir**; ama her hatalı erişim segfault vermez: dizinin bir ötesine yazmak çoğu zaman aynı sayfada, izinli bir yere düşer ve **sessizce** başka bir veriyi bozar (H10, HA7).

3.9 Dinamik bellek: malloc, calloc, realloc, free

Dizinin boyutu derleme zamanında bilinmiyorsa (kaç öğrenci gelecek?) bellek **çalışma zamanında** öbekten istenir. `stdlib.h`'deki dört fonksiyon Tablo 12.2'dedir (ISO/IEC, 2018, 7.22.3; King, 2008, Bölüm 17).

Tablo 12.2
Dinamik bellek fonksiyonları (stdlib.h)

Fonksiyon	Ne yapar?	Başarısızlıkta	Not
`malloc(boyut)`	boyut baytlık blok ayırır; içerik belirsizdir	NULL	boyutu `n * sizeof *p` ile verin
`calloc(adet, boyut)`	adet × boyut baytlık blok ayırır, bütün baytları sıfırlar	NULL	sayaç dizileri için doğal seçim (H12_K08)
`realloc(p, yeni_boyut)`	bloğu büyütür/küçültür; gerekirse yeni yere taşıyıp içeriği kopyalar	NULL; eski blok yerinde ve geçerli kalır	sonucu GEÇİCİ işaretçiye alın
`free(p)`	bloğu geri verir	—	`free(NULL)` etkisizdir; sonra p = NULL

Dört kural: (1) `malloc`'un `void *` dönüşü her nesne işaretçisine **kendiliğinden** dönüşür; C'de dönüştürme (cast) **gerekmez** (C++'ta gerekir). (2) Boyutu `n * sizeof *p` ile yazın; `sizeof p` işaretçinin 8 baytıdır (HA7). (3) Dönüşü **her zaman** NULL'a karşı denetleyin. (4) Her bloğun **tek sahibi** onu **tam bir kez** bırakıp işaretçiyi NULL yapsın; yaşam döngüsü NULL → ayır → kullan → bırak → NULL'dur.

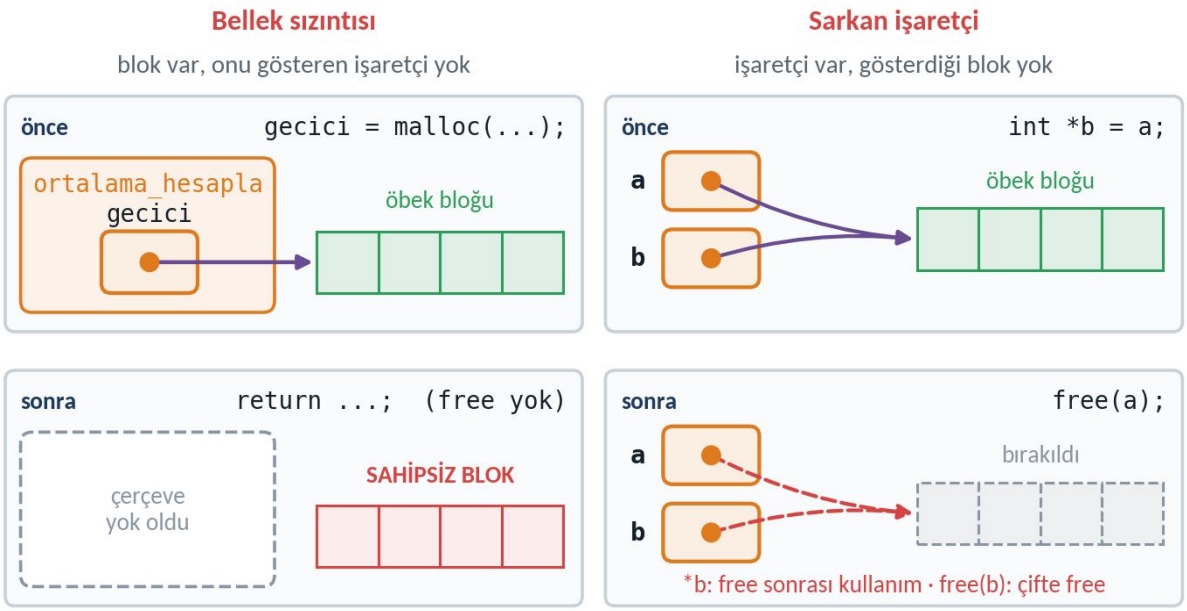
Sıfır boyut. `malloc(0)`'ın NULL mü yoksa kullanılamayacak bir işaretçi mi döndüreceği **uygulamaya bağlıdır** (ISO/IEC, 2018, 7.22.3); `realloc(p, 0)`'ın eski bloğu bırakıp bırakmayacağı da C17'de uygulamaya bağlıdır. Taşınabilir kod sıfır boyutlu istek yapmaz. **Başarısızlık gerçektir:** karşılanması olanaksız bir boyut (yaklaşık 8 eksabayt) isteyen `H12_K09_bellek_yetersiz.c`'de iki fonksiyon da NULL döndürdü ve `realloc` başarısız olunca eski blok yerinde kaldı (gerçek çıktı: "malloc(9223372036854775807) -> NULL", "realloc başarısız; eski blok yerinde: 10 20 30").

`dizi = realloc(dizi, yeni)` yazmak tehlikelidir: realloc NULL döndürürse eski bloğun **tek** adresi üzerine NULL yazılır, blok sahipsiz kalır (sızıntı) ve veri kaybolur. **calloc** ile sayaç dizisi (H10'daki frekans deseni, bu kez öbekte): `H12_K08_calloc_sifir.c` sekiz zar değerini saydı; ilk iki satır "1 değeri 2 kez", "2 değeri 0 kez".

3.10 Bellek hataları, AddressSanitizer ve Valgrind

Dinamik belleğin iki temel hatası birbirinin aynadaki görüntüsüdür (Şekil 12.9). **Bellek sızıntısında** (memory leak) blok vardır ama onu gösteren işaretçi kalmamıştır: bırakılamaz, program bitene dek yer kaplar. **Sarkan işaretçide** (dangling pointer) işaretçi vardır ama gösterdiği blok bırakılmıştır (ya da çerçevesi yok olmuştur): onu kullanmak **free sonrası kullanım**, yeniden bırakmak **çifte free**'dir.

Şekil 12.9
Sızıntı (solda: çerçeve yok oldu, blok sahipsiz) ve sarkan işaretçi (sağda: blok bırakıldı, a ve b hâlâ onu gösteriyor)



Bu hatalar çoğu zaman **sessizdir** (Örnek 13'ün sınır dışı yazma ve sızıntı seçenekleri ASan'sız "normal" bitti). İki araç onları görünür kılar. **AddressSanitizer** (ASan) derleyiciye `-fsanitize=address` ile eklenir; her bellek erişimini denetleyen kod yerleştirir ve hatayı **ilk** gerçekleştiği satırda, bloğun nerede ayrıldığı ve bırakıldığı bilgisiyle raporlar (Serebryany vd., 2012); içindeki **LeakSanitizer** program bitince sızıntıları listeler. **Valgrind** (Memcheck) ise programı yeniden derlemeden, sanal bir işlemcide çalıştırarak denetler (Nethercote & Seward, 2007); çok daha yavaştır ama başlatılmamış değerlerin kullanımını da yakalar (HA3). Bir ASan raporunun okunuşu Bölüm 5'teki Şekil 12.10'tedir. ASan başlatılmamış değer okumayı, Valgrind yığındaki dizilerin taşmasını yakalayamaz; ikisi birbirini tamamlar.

4. Çözümlü Örnekler: Örnek Merdiveni

Örnekler kolaydan zora sıralıdır (`kodlar/H12_01_…` – `H12_14_…`); her biri bir yeni öge ekler. Kasıtlı hatalı Örnek 13 dışında hepsi **sıfır uyarıyla** derlenir ve ASan + UBSan altında temiz çalışır; çıktılar gerçektir. Örnek 1, 2 ve 7 Bölüm 3'tedir; **Örnek 3** ve **Örnek 12** tam şablonla işlenir.

4.1 Amiral örnek — Örnek 3 ★★: referansla takas

1. Problem. İki tamsayı değişkenin değerlerini bir fonksiyonla değiş tokuş edin; çağrıdan sonra `main`'deki değişkenler gerçekten değişmiş olsun. H8'deki başarısız sürümle (`takas_deger`) yan yana çalıştırıp farkı adreslerle kanıtlayın.

Tablo 12.3
Örnek 3'ün girdi–işlem–çıktı çözümlemesi

GİRDİ	İŞLEM	ÇIKTI
• x ve y'nin ADRESLERİ (`int *a`, `int *b`)	• gecici ← *a • *a ← *b • *b ← gecici	• x ile y'nin değerleri değiş tokuş edilmiş • adresler (kanıt için)

2–3. Çözümleme ve algoritma. Fonksiyon x ve y'nin kendisine ancak **adresleri** üzerinden ulaşabilir. Algoritma H3'teki geçici değişkenli takasın aynısıdır; yalnızca `x` yerine `*a`, `y` yerine `*b` yazılır. Sınır durumu: `takas(&t, &t)` de doğru çalışır. **4. C kodu:**

H12_03_takas.c · satır 21–39

```
21  /* H12: a ve b ADRES taşır; *a ve *b main'deki kutulardır */
22  void takas(int *a, int *b)
23  {
24      printf("  takas      :  a = %p,  b = %p\n", (void *)a, (void *)b);
25      int gecici = *a;
26      *a = *b;
27      *b = gecici;
28  }
29
30  int main(void)
31  {
32      int x = 3, y = 7;
33      printf("  main       : &x = %p, &y = %p\n", (void *)&x, (void *)&y);
34      takas_deger(x, y);
35      printf("takas_deger'den : x = %d, y = %d   <- değişmedi\n", x, y);
36      takas(&x, &y);
37      printf("takas'tan sonra : x = %d, y = %d   <- değişti\n", x, y);
38      return 0;
39  }
```

5. Satır satır. Satır 13–19 (gösterilmedi): H8'deki sürüm, `takas_deger`; a ve b, x ile y'nin **kopyalarıdır** (adresleri farklı). Satır 22: parametreler adres taşır. Satır 25–27: her dolaylama main'in kutularına uzanır. Satır 36: çağıran, kutuların adreslerini verir; `takas(x, y)` yazılsaydı GCC 13 `int`'ten `int *`'a dönüşümü -Wint-conversion ile uyarırdı (GCC 14'ten itibaren hata).

6. İzleme. `takas(&x, &y)` çağrısının içinde (a ve b, x ve y'yi gösterir):

Tablo 12.4
takas(&x, &y) izleme tablosu: değişen hücre vurgulu

Adım	Satır	Deyim	*a (= x)	*b (= y)	gecici
0	36	çağrı	3	7	–
1	25	`int gecici = *a;`	3	7	**3**
2	26	`*a = *b;`	**7**	7	3
3	27	`*b = gecici;`	7	**3**	3

7. Örnek çalıştırma (gerçek):

GERÇEK ÇIKTI

```
$ setarch x86_64 -R ./takas      # ASLR kapalı (takas şeklindeki adresler)
  main       : &x = 0x7fffffff8f20, &y = 0x7fffffff8f24
  takas_deger: &a = 0x7fffffff8efc, &b = 0x7fffffff8ef8
takas_deger'den : x = 3, y = 7   <- değişmedi
  takas      :  a = 0x7fffffff8f20,  b = 0x7fffffff8f24
takas'tan sonra : x = 7, y = 3   <- değişti
```

Kanıt adreslerdedir: `takas_deger`'in `&a`'sı (`0x7fffffff8efc`) x'in adresinden (`0x7fffffff8f20`) **farklıdır**: başka bir kutu, başka bir çerçeve. `takas`'ın `a`'sı ise x'in adresinin **ta kendisidir**. Yığın aşağı büyüdüğü için çağrılanın çerçevesi main'in altındaki adreslerdedir (H9).

8. Test. `H12_K14_takas_testleri.c` takas'ı normal (3, 7), eşit değerli (5, 5), negatif (−2, 40) ve aynı değişkenin iki kez verildiği `takas(&t, &t)` durumlarında `assert` ile sınar; gerçek çıktı: "takas: 4 test geçti.". **9. Değiştir.** (a) Üç değişkeni döndüren `dondur(&a, &b, &c)` yazın (a ← b ← c ← a). (b) Geçici değişken yerine `*a ^= *b; *b ^= *a; *a ^= *b;` yazın: `takas(&t, &t)` neden t'yi 0 yapar? (c) takas, H10'un ters çevirmesinde (Örnek 8) ve AP-2'deki bütün yerinde sıralama algoritmalarında temel işlemdir.

4.2 Örnek 4–6: çıktı parametreleri ve p++

Örnek 4 ★★ — bolum_kalan: birden çok sonuç

H12_04_bolum_kalan.c · satır 18–26

```c
bool bolum_kalan(int a, int b, int *bolum, int *kalan)
{
    if (b == 0) {
        return false;
    }
    *bolum = a / b;
    *kalan = a % b;
    return true;
}
```

Dönüş değeri **başarıyı**, çıktı parametreleri **sonuçları** taşır; `main` girdiyi `scanf("%d %d", &a, &b)` ile okur ve `bolum_kalan(a, b, &bolum, &kalan)` çağırır. Gerçek çalıştırmalar: "17 5" için "17 = 5 * 3 + 2", "-17 5" için "-17 = 5 * -3 + -2", "7 0" için "Sıfıra bölme: bölüm ve kalan tanımsız." (b = 0 iken fonksiyon çıktılara dokunmaz, çağıran onları okumaz). C17'de tamsayı bölmesi sıfıra doğru keser: her durumda a == b × (a / b) + a % b korunur. **Değiştir.** `INT_MIN / -1` taşar (tanımsız davranış): bu durumu da `false` ile reddedin.

Örnek 5 ★★ — dizi_min_max

H12_05_min_max.c · satır 13–25

```c
void dizi_min_max(const int dizi[], int n, int *min, int *max)
{
    *min = dizi[0];
    *max = dizi[0];
    for (int i = 1; i < n; i++) {
        if (dizi[i] < *min) {
            *min = dizi[i];
        }
        if (dizi[i] > *max) {
            *max = dizi[i];
        }
    }
}
```

Fonksiyon diziyi **tek geçişte** dolaşıp iki sonucu birlikte bulur; ara sonuçları doğrudan çağıranın kutularında (`*min`, `*max`) günceller. `const int dizi[]`, diziyi değiştirmeme sözüdür. Ön koşul n ≥ 1; tek elemanlı dizi sınır durumudur (gerçek çıktı: "6 not: en küçük 38, en büyük 90, aralık 52", "1 not: en

küçük 50, en büyük 50"). **Değiştir.** En küçüğün ve en büyüğün **indislerini** de döndürün (Mini-OBS'deki `min_max_indis`).

Örnek 6 ★★ — p++ ile dizi dolaşma

H12_06_isaretci_aritmetigi.c · satır 15–23

```
15      int a[5] = {10, 20, 30, 40, 50};
16      int *son = a + 5;                   /* son elemanın BİR ötesi */
17      int toplam = 0;
18
19      for (int *p = a; p < son; p++) {    /* p++ : bir sonraki int */
20          printf("p = %p   *p = %d   p - a = %td\n", (void *)p, *p, p - a);
21          toplam += *p;
22      }
23      printf("toplam = %d\n", toplam);
```

Döngü değişkeni bir **indis** değil bir **adrestir**; her `p++` adresi 4 bayt ilerletir (gerçek çalıştırmada p 0x…fd80, 0x…fd84, 0x…fd88 diye 4'er arttı; Şekil 12.6) ve `p - a` kaçıncı elemanda olduğumuzu verir. `son = a + 5` dizinin bir ötesidir: `p < son` karşılaştırması güvenlidir, `*son` tanımsız davranış olurdu. Program "toplam = 150" yazar. **Değiştir.** Aynı döngüyü `while (p < son) toplam += *p++;` biçiminde yazın; `*p++`'nın neden doğru çalıştığını öncelikle açıklayın.

4.3 Örnek 8–11: iki işaretçi, const, malloc, baytlar

Örnek 8 ★★ — İki işaretçiyle ters çevirme

H12_08_ters_cevir_isaretci.c · satır 30–38

```
30  void ters_cevir(int *bas, int *son)
31  {
32      while (bas < son) {
33          printf("  takas: *bas = %d, *son = %d\n", *bas, *son);
34          takas(bas, son);
35          bas++;
36          son--;
37      }
38  }
```

H10'daki `H10_07_ters_cevir.c` iki **indisle** (i baştan, j sondan) çalışıyordu; aynı dosyadaki `ters_cevir_indis` bunun kopyasıdır ve `takas(&a[i], &a[j])` çağırır. İşaretçili sürümün kazancı, **herhangi bir alt diziye** uygulanabilmesidir: `ters_cevir(a + 1, a + n - 2)` yalnızca ortadaki beş elemanı çevirdi (gerçek çıktı: "1 2 3 4 5 6 7" → "1 6 5 4 3 2 7"). Bu iki işaretçi tekniği AP-2'de "two pointers" yöntemine dönüşür.

Örnek 9 ★★ — const işaretçiler

`H12_09_const_isaretci.c` üç bildirimin geçerli kullanımlarını (`p = &y;`, `*q = 10;`) ve `const int *a` parametreli bir `toplam` fonksiyonunu içerir (Şekil 12.7; gerçek çıktı "*p = 2, x = 10, *r = 2", "toplam = 108"); yasak satırların derleyici iletileri Bölüm 3.7'dedir. `toplam(const int *a, int n)` gibi salt okuyan her dizi fonksiyonu parametresini `const` yapmalıdır: hem okuyucuya hem derleyiciye sözleşmeyi söyler (H10'daki `const` dizi parametresi aynı şeydir).

Örnek 10 ★★ — malloc ile not dizisi

H12_10_malloc_not_dizisi.c · satır 23–37

```
23      double *notlar = malloc(n * sizeof *notlar);    /* öbekten n double */
24      if (notlar == NULL) {                           /* her zaman denetle */
25          printf("Bellek yetersiz.\n");
26          return 1;
27      }
28
29      double toplam = 0.0;
30      for (int i = 0; i < n; i++) {
31          printf("%d. not: ", i + 1);
32          if (scanf("%lf", &notlar[i]) != 1) {
33              printf("Geçersiz not.\n");
34              free(notlar);                           /* erken çıkışta da */
35              return 1;
36          }
37          toplam += notlar[i];
```

İşaretçi `notlar` **yığında**, gösterdiği blok **öbektedir**: gerçek çalıştırmanın (4 not, "Ortalama: 75.00") son satırı "notlar (yığında) 0x7fff604a9870 -> blok (öbekte) 0x55985adcaac0" iki farklı bölgeyi gösterir. Program sonunda `free(notlar); notlar = NULL;` yazılır. Erken çıkış yollarında da `free` çağrılır ("elli" girdisiyle denendi; bütün çalıştırmalar `ciktilar/` altında). Valgrind kanıtı: "total heap usage: 3 allocs, 3 frees, 8,224 bytes allocated", "All heap blocks were freed -- no leaks are possible", "ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)"; üç ayırmanın biri bizim bloğumuz, ikisi `stdio`'nun girdi/çıktı tamponlarıdır.

Örnek 11 ★★★ — char * ile bir int'in baytları

H12_11_bayt_inceleme.c · satır 13–30

```
13  #include <stdio.h>
14
15  int main(void)
16  {
17      int sayi = 0x12345678;                      /* H03_K02 ile aynı sayı */
18      unsigned char *b = (unsigned char *)&sayi;  /* bayt bayt bakar */
19
20      printf("sayi = 0x%08X, sizeof sayi = %zu\n", (unsigned)sayi, sizeof sayi);
21      for (size_t i = 0; i < sizeof sayi; i++) {
22          printf("  b + %zu = %p : %02X\n", i, (void *)(b + i), b[i]);
23      }
24      printf("İlk bayt 0x%02X: bu makine %s-uçlu.\n",
25             b[0], b[0] == 0x78 ? "küçük" : "büyük");
26
27      b[3] = 0x7F;                                /* en anlamlı bayt */
28      printf("b[3] = 0x7F sonrası sayi = 0x%08X\n", (unsigned)sayi);
29      return 0;
30  }
```

H3'te (`H03_K02_bayt_sirasi.c`) aynı sayının baytlarını gdb ile görmüştük; şimdi C'nin içinden okuyoruz: x86-64 **küçük-uçludur** (little-endian), 0x12345678'in baytları bellekte 78 56 34 12 sırasındadır ve `b + k` bir bayt ilerler. Bir nesnenin baytlarına **karakter tipi** işaretçisiyle erişmek C'de izinli olan tek "tip değiştirerek bakma" yoludur; `unsigned char` seçilir çünkü 0x80 ve üzeri baytlar `char`

işaretliyse negatif görünür. Program “İlk bayt 0x78: bu makine küçük-uçlu.”, “b[3] = 0x7F sonrası sayi = 0x7F345678” yazdı: `b[3] = 0x7F` en anlamlı baytı değiştirdi. Ağ protokolleri ve dosya biçimleri bayt sırasını sabitler; bu yüzden ikili veri okurken bayt sırası bilinmelidir (H14).

4.4 Amiral örnek — Örnek 12 ★★★: realloc ile büyüyen dizi

1. Problem. Kaç eleman geleceği önceden bilinmeyen bir veri akışını (burada 1’den 20’ye kadar sayıların kareleri) bir dizide toplayın; dizi dolunca kapasitesini **ikiye katlayın**, her büyütmede bloğun yerinde mi büyüdüğünü yoksa taşındığını mı raporlayın.

Tablo 12.5
Örnek 12’nin girdi–işlem–çıktı çözümlemesi

GİRDİ	İŞLEM	ÇIKTI
• sırayla gelen değerler (20 tane) • başlangıç kapasitesi 1	• n == kapasite ise realloc ile 2 × kapasite • sonucu geçici işaretçiye al, NULL’ı denetle • dizi[n] = değer; n++	• 20 elemanlık dizi • kapasite adımları ve taşınma sayısı

2–3. Çözümleme ve algoritma. Her elemanda yeniden ayırmak çok sayıda kopya demektir; ikiye katlamada büyütme sayısı yalnızca $\log_2 n$ kadardır. Sınırlar: realloc NULL dönerse eski blok geçerli kalmalı; `2 * kapasite` taşmamalı. Sözde kod: `n == kapasite ise { gecici ← realloc(dizi, 2·kapasite·4); gecici = NULL ise dur; dizi ← gecici; kapasite ← 2·kapasite }; dizi[n] ← değer; n ← n + 1`. **4. C kodu:**

H12_12_realloc_dinamik_dizi.c · satır 15–49

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>

int main(void)
{
    int kapasite = 1, n = 0, tasinma = 0;
    int *dizi = malloc(kapasite * sizeof *dizi);
    if (dizi == NULL) {
        return 1;
    }

    for (int deger = 1; deger <= 20; deger++) {
        if (n == kapasite) {                    /* dolu: ikiye katla */
            uintptr_t eski = (uintptr_t)dizi;   /* adresi ÖNCE sakla */
            int *gecici = realloc(dizi, 2 * kapasite * sizeof *dizi);
            if (gecici == NULL) {               /* dizi hâlâ geçerli */
                printf("Bellek yetersiz: %d elemanda kaldı.\n", n);
                break;
            }
            dizi = gecici;
            kapasite *= 2;
            int tasindi = (uintptr_t)dizi != eski;
            tasinma += tasindi;
            printf("n = %2d: kapasite %2d -> %2d  %s\n", n, kapasite / 2,
                   kapasite, tasindi ? "(blok taşındı)" : "(yerinde büyüdü)");
```

```
41          }
42          dizi[n] = deger * deger;
43          n++;
44      }
45      printf("%d eleman, kapasite %d, %d taşınma; son eleman %d\n",
46             n, kapasite, tasinma, dizi[n - 1]);
47      free(dizi);
48      return 0;
49  }
```

5. Satır satır. Satır 29: taşınmayı anlamak için eski adres **realloc'tan önce** tamsayı olarak saklanır; başarılı bir realloc'tan sonra eski işaretçinin değeri belirsizdir ve hiçbir biçimde kullanılamaz (karşılaştırma bile). Satır 30: sonuç **geçici** işaretçiye alınır. Satır 31: başarısızlıkta `dizi` hâlâ eski bloğu gösterir; veri kaybolmaz, döngü durur. Satır 35: yalnızca başarıda bağlanır. Satır 47: tek blok, tek `free`.

Tablo 12.6
Örnek 12'nin kapasite izleme tablosu (gerçek çalıştırmadan)

Büyütme	n (dolu)	kapasite önce → sonra	Blok	Kopyalanan eleman (en çok)
1	1	1 → 2	yerinde büyüdü	1
2	2	2 → 4	yerinde büyüdü	2
3	4	4 → 8	**taşındı**	4
4	8	8 → 16	yerinde büyüdü	8
5	16	16 → 32	yerinde büyüdü	16

7. Örnek çalıştırma (gerçek):

ÇIKTI
```
$ ./realloc_dizi
n =  1: kapasite  1 ->  2  (yerinde büyüdü)
n =  2: kapasite  2 ->  4  (yerinde büyüdü)
n =  4: kapasite  4 ->  8  (blok taşındı)
n =  8: kapasite  8 -> 16  (yerinde büyüdü)
n = 16: kapasite 16 -> 32  (yerinde büyüdü)
20 eleman, kapasite 32, 1 taşınma; son eleman 400
```

8. Test ve bir gözlem. 20 eleman 5 büyütmeyle sığdı; en kötü durumda kopyalanan eleman sayısı 1 + 2 + 4 + 8 + 16 = 31, yani eleman sayısının iki katından az. **Aynı program Valgrind altında** çalıştırıldığında ise beş büyütmenin **beşinde de** blok taşındı ("20 eleman, kapasite 32, 5 taşınma; son eleman 400"): Valgrind'in bellek ayırıcısı her realloc'ta yeni blok verir. Yani "yerinde büyüme" bir **garanti değildir**; ayırıcıya bağlıdır ve kod her iki duruma da hazırlıklı olmalıdır: realloc'tan sonra yalnızca **dönen** işaretçi kullanılır. Valgrind özeti: "total heap usage: 7 allocs, 7 frees, 4,348 bytes allocated", "All heap blocks were freed".

9. Değiştir. (a) Sayıları `scanf` ile EOF'a kadar okuyun (Alıştırma 12.10). (b) Kapasiteyi ikiye katlamak yerine 1'er artırın ve realloc sayısını karşılaştırın. (c) Büyütmeyi bir fonksiyona taşıyın: `int ekle(int **dizi, int *n, int *kapasite, int deger)` neden `int **` ister? (Merak Kutusu, Bölüm 10).

4.5 Örnek 13–14: hata avı ve fonksiyon işaretçileri

Örnek 13 ★★★ — AddressSanitizer ile hata avı

`H12_13_asan_hatalari.c` beş küçük fonksiyonun her birinde bilerek bir bellek hatası yapar (ör. `sinir_disi`, 4 elemanlık bloğa `a[4] = 99;` yazar); seçim girdiden okunur (dosya "BU DOSYA KASITLI HATA İÇERİR" işaretlidir). Program bir kez `-fsanitize=address` ile, bir kez onsuz derlendi; beş seçenek iki sürümle de çalıştırıldı (Tablo 12.7; raporların tamamı `kodlar/ciktilar/H12_13_asan_hatalari.txt`'dedir).

Tablo 12.7
Örnek 13: aynı beş hata AddressSanitizer'lı ve ASan'sız derlemede (gerçek çalıştırmalar)

Seçim	Hata	ASan'lı derleme: rapor başlığı	ASan'sız derleme: gözlenen	Çıkış (ASan'sız)
1	sınır dışı yazma (`a[4]`)	AddressSanitizer: heap-buffer-overflow	Program sona erdi (seçim 1).	0
2	free sonrası okuma	AddressSanitizer: heap-use-after-free	*p = 1624208527 · Program sona erdi (seçim 2).	0
3	çifte free	AddressSanitizer: attempting double-free	free(): double free detected in tcache 2 · kabuk: Aborted	134
4	sızıntı (400 bayt)	LeakSanitizer: detected memory leaks	Program sona erdi (seçim 4).	0
5	NULL'a yazma	AddressSanitizer: SEGV	kabuk: Segmentation fault	139

Tablonun dersi: ASan'sız derlemede beş hatanın **ikisi hiç belli olmadı** (sınır dışı yazma ve sızıntı çıkış durumu 0 ile bitti), free sonrası okuma **anlamsız bir değer** yazdı; yalnızca çifte free (glibc'nin kendi denetimi, `Aborted`, 134) ve NULL'a yazma (segfault, 139) programı durdurdu. ASan beşini de dosya ve satır numarasıyla yakaladı: dördünü **hatanın olduğu anda**, sızıntıyı ise LeakSanitizer **program biterken** (ASan'lı programın çıkış durumu 1). Derleyici bile iki hatayı derleme zamanında uyardı (`-Wuse-after-free`): uyarıları okumak ilk savunma hattıdır.

Örnek 14 ★★★ — Fonksiyon işaretçileri (meydan okuma)

H12_14_fonksiyon_isaretcisi.c · satır 41–48

```
41  int main(void)
42  {
43      double (*islemler[])(double, double) = {topla, cikar, carp, bol};
44      const char isaretler[] = {'+', '-', '*', '/'};
45      int adet = (int)(sizeof islemler / sizeof islemler[0]);
46
47      for (int i = 0; i < adet; i++) {            /* tabloyu dolaş */
48          printf("12 %c 4 = %.2f\n", isaretler[i], islemler[i](12, 4));
```

Fonksiyonlar da bellekte (text bölgesinde) durur ve adresleri vardır. `double (*islem)(double, double)` bildirimini içten dışa okuyun: "islem, iki `double` alıp `double` döndüren bir fonksiyonu gösteren işaretçidir" (parantez şarttır; `double *islem(…)` işaretçi **döndüren fonksiyon** olurdu). `islemler` bu işaretçilerden oluşan bir dizidir; `switch` yerine işaret karakteriyle bulunan **satır** çağrılır (gerçek çıktı "12 + 4 = 16.00" … "12 / 4 = 3.00"; girdi "7 * 6" için "7 * 6 = 42.00"). Aynı dosyadaki `double uygula(double (*islem)(double, double), double a, double b)` ise davranışı

parametre olarak alır (`return islem(a, b);`): AP-2'deki `qsort`'un karşılaştırma fonksiyonu bu fikirle çalışır.

5. Hata Avcısı

Dokuz klasik işaretçi hatası; her biri `kodlar/hata_avcisi/` altında kasıtlı hatalı ve düzeltilmiş iki dosya olarak. İletiler ve çıktılar gerçektir; ASan/Valgrind raporları kısaltılmıştır (tamamı `hata_avcisi/ciktilar/` altında). Özet Tablo 12.8'te, bir ASan raporunun parça parça okunuşu Şekil 12.10'tedir.

Tablo 12.8
Hata Avcısı özeti: hata, aracın gerçek iletisi ve düzeltme

HA	Hata	Araç / ileti (gerçek, kısaltılmış)	Düzeltme
1	`*` ile `&` karıştırma: `int y = p;, p = x;`	uyarı: initialization of 'int' from 'int *' makes integer from pointer without a cast ; çalışınca segfault (139)	`y = *p;, p = &x;`
2	`int *p, q;`: q işaretçi değil	uyarı: assignment to 'int' from 'int *' makes integer from pointer without a cast ; sizeof q = 4	`int *p, *q;` ya da ayrı satırlar
3	başlatılmamış işaretçiye yazma	uyarı: 'p' is used uninitialized [-Wuninitialized]; program "çalıştı" (0); Valgrind: Use of uninitialised value	`int *p = NULL;` … geçerli adres
4	NULL'ı dolaylama	segfault (139); ASan: SEGV	dönüşü NULL'a karşı denetle
5	free sonrası kullanım	uyarı: pointer 'notlar' used after 'free' ; ASan: heap-use-after-free	önce kullan, sonra free; p = NULL
6	çifte free (iki işaretçi, tek blok)	glibc: free(): double free detected (134); ASan: attempting double-free	tek sahip; free sonrası bütün kopyalar NULL
7	`malloc(sizeof(kaynak))`, NULL denetimi yok	uyarı yok; ASan'sız sessiz; ASan: heap-buffer-overflow	`malloc(n * sizeof *yeni)` + NULL denetimi
8	yerel değişkenin adresini döndürme	uyarı: function returns address of local variable ; GCC NULL döndürdü → segfault	öbekte ayır (çağıran free eder) ya da diziyi çağıran versin
9	bellek sızıntısı	çıktı doğru; LeakSanitizer: 12000 bayt / 3 blok; Valgrind: definitely lost	dönmeden önce `free(gecici)`

Tablonun gizledikleri

Uyarı yeter mi? HA1'in `int y = p;` ve `p = x;` satırları yalnızca **uyarı** aldı (GCC 13'te `-Wint-conversion` uyarıdır, GCC 14'ten itibaren hatadır); çalışınca `*p`, 0x5 adresini okumaya çalıştı ve süreç SIGSEGV aldı. **Sessiz bozulma.** HA3 bu çalıştırmada çökmedi ("*p = 10", çıkış 0): çöp adres yazılabilir bir yere denk geldi, oradaki veri bozuldu; yalnızca Valgrind yakaladı. ASan'sız HA5 "ilk not: 0.0" yazdı (70 değil: glibc bırakılan bloğun ilk baytlarını kendi kayıtları için kullanır), HA7 ise doğru sonucu yazdı. **Derleyici bile şaşırtır.** HA8'de GCC yerel adres yerine **NULL** döndürdü ("k = (nil)"); başka bir derleyicide program bir süre "doğru" görünebilir. Kök neden çoğu zaman **sahiplik** belirsizliğidir: bloğu yalnızca sahibi bırakmalı, bırakınca bütün kopyalar NULL yapılmalıdır.

Şekil 12.10
AddressSanitizer raporunu okumak: hata türü, erişim, nerede erişildi, hangi blok, nerede bırakıldı, nerede ayrıldı (HA5'in gerçek raporu)

AddressSanitizer raporunu okumak (H12_HA5_free_sonrasi.c, kısaltılmış gerçek rapor)

```
ERROR: AddressSanitizer: heap-use-after-free

READ of size 8 at 0x503000000040 thread T0

  #0 ... in main H12_HA5_free_sonrasi.c:21

0x… is located 0 bytes inside of 24-byte region […]

freed by thread T0 here:

  #1 ... in main H12_HA5_free_sonrasi.c:20

previously allocated by thread T0 here:

  #1 ... in main H12_HA5_free_sonrasi.c:13

SUMMARY: AddressSanitizer: heap-use-after-free …
```

1 hata türü
2 erişim: okuma, 8 bayt
3 NEREDE erişildi
4 hangi blok
5 NEREDE bırakıldı
6 NEREDE ayrıldı
7 özet

HA9 · GERÇEK SIZINTI RAPORLARI (KISALTILMIŞ)

```
$ ./ha9_asan      # LeakSanitizer
Direct leak of 12000 byte(s) in 3 object(s) allocated from:
    #1 0x55778390a248 in ortalama_hesapla H12_HA9_sizinti.c:13
SUMMARY: AddressSanitizer: 12000 byte(s) leaked in 3 allocation(s).
$ valgrind --leak-check=full ./ha9      # seçilmiş satır
==29501== 12,000 bytes in 3 blocks are definitely lost in loss record 1 of 1
```

HA9. `ortalama_hesapla` her çağrıda 1000 `int`'lik blok ayırır ve `free` etmeden döner; program doğru sonucu ("sonuç: 1498.5") yazar, sızıntı ancak araçla görünür (üç çağrıda 12 000 bayt; iki araç da bloğun ayrıldığı satırı gösterir). Kural: **her malloc'un tam olarak bir free'si**.

6. Mühendis Gözüyle: Sahiplik, Araçlar ve Bellek Güvenliği

İşaretçi hataları **derleyicinin göremediği**, programın da çoğu zaman haber vermediği hatalardır. Profesyonel C kodunda üç alışkanlık riski büyük ölçüde azaltır:

- **Sahiplik (ownership).** Her öbek bloğunun tek bir sahibi vardır ve bloğu yalnızca o bırakır; blok döndüren bir fonksiyonda (HA8'in düzeltilmiş sürümü, Alıştırma 12.8) bırakma sorumluluğu prototipin yanına yazılır.
- **Tek temizlik yolu.** Erken `return`'ler bırakılmamış blok bırakır; Mini-OBS v0.12'de bütün çıkışlar tek bir `kayitlari_birak(...)` çağrısından geçer.
- **Araçla doğrulama.** Bellek kullanan her değişiklik `-fsanitize=address,undefined` ve (Linux/WSL'de) `valgrind --leak-check=full` ile denenir; Mini-OBS'nin `make asan` ve `make valgrind` hedefleri bunu tek komuta indirir.

Bu haftanın hataları SEI CERT C kodlama standardının kurallarıyla doğrudan eşleşir (Carnegie Mellon University Software Engineering Institute, 2016; Seacord, 2013): **EXP33-C** başlatılmamış belleği

okumayın (HA3); **EXP34-C** NULL işaretçiyi dolaylamayın (HA4, K07); **ARR30-C** sınır dışı işaretçi ya da indis kullanmayın (Örnek 13/1; Örnek 6'da `a + 5` yalnızca karşılaştırılır); **MEM30-C** bırakılmış belleğe erişmeyin (HA5); **MEM31-C** gerekmeyen dinamik belleği bırakın (HA9, `kayitlari_birak`); **MEM35-C** nesneye yeterli bellek ayırın (HA7); **DCL30-C** nesneleri uygun saklama süresiyle bildirin (HA8); **ERR33-C** kütüphane hatalarını algılayın (`malloc`/`realloc` NULL denetimi, K09).

Tanımsız davranış öngörülemez: başka derleyici, `-O2` ya da başka makine farklı davranabilir; "bende çalışıyor" kanıt değildir. **Taşınabilirlik:** MinGW-w64'ün GCC'si ASan'ı desteklemez (WSL ya da Visual Studio'nun `/fsanitize=address` seçeneği kullanılır); Valgrind yalnızca Linux'ta (WSL dahil) çalışır. **Verimlilik:** dinamik dizi elemanları **bitişik** tutar; kapasiteyi ikiye katlamak eleman başına ortalama sabit sayıda kopya demektir (Örnek 12: 20 eleman, en çok 31 kopya), birer artırmak ise yaklaşık $n^2/2$ kopya (**amortize analiz**, AP-2).

7. Mini-OBS v0.12: Dinamik Diziler ve İşaretçi Parametreler

v0.11'de kayıtlar `main`'in yığınındaki sabit dizilerdeydi (`AZAMI_OGRENCI` = 100): 101. öğrenci giremiyor, 3 öğrenci için 100 satır ayrılıyordu. v0.12'de **kayıtlar öbekte** tutulur: kapasite çalışma zamanında sorulur, dolunca `realloc` ile ikiye katlanır; istatistikler **işaretçi parametrelerle** döner; bütün çıkışlar tek temizlik yolundan geçer. Hesap kuralları (RTEÜ harf tablosu, baraj, şartlı DC, iki haneye yuvarlanan YANO), menü ve raporlar **aynıdır**; testler 61'den 79 doğrulamaya çıktı.

Tasarım kararı: satır işaretçisi. Notlar tek boyutlu bir `double *` blokta tutulup H11'in formülüyle (`vize[i * DERS_SAYISI + d]`) de okunabilirdi; biz `double (*vize)[DERS_SAYISI]`, yani **satırları gösteren** bir işaretçi seçtik: v0.11'in `double m[][DERS_SAYISI]` parametreli bütün fonksiyonları (bu parametre zaten `double (*m)[DERS_SAYISI]` demektir, Bölüm 3.6) ve `vize[i][d]` yazımları **hiç değişmeden** çalışır.

`main`, kapasiteyi okuduktan sonra dört bloğu ayrı ayrı ister (bellek düzeni Şekil 12.11'te): `long long *no = malloc(kapasite * sizeof *no);` ve `double (*vize)[DERS_SAYISI] = malloc(kapasite * sizeof *vize);` (final ve ort için aynısı). Herhangi biri NULL dönerse **hepsi** `kayitlari_birak(no, vize, final, ort)` ile bırakılır: `free(NULL)` etkisiz olduğundan başarılı olanları ayırt etmeye gerek yoktur. Kapasite dolunca `main` dört bloğu **tek tek** büyütür; `obs.c`'deki iki küçük yardımcıdan `yeni_kapasite` iki katını (`int`'e sığmazsa −1), `buyut` ise `realloc(blok, kapasite × eleman_boyutu)` sonucunu döndürür:

proje/main.c · satır 51–62

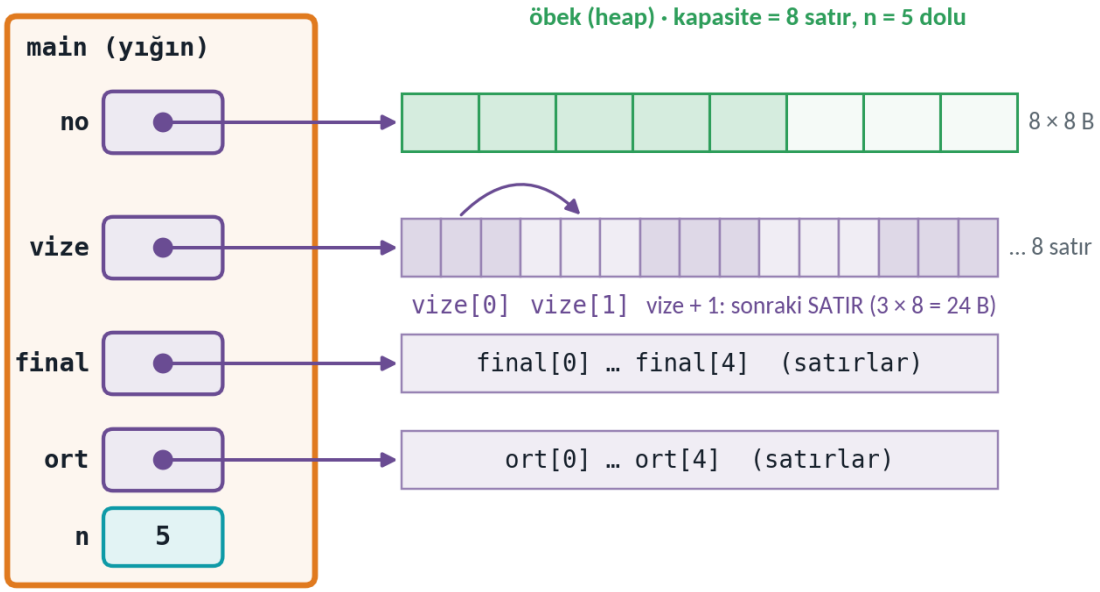
```
51              if (n == kapasite) {              /* dolu: dört blok 2 katına */
52                  int yeni = yeni_kapasite(kapasite);   /* taşmada -1 */
53                  bool tamam = (yeni != -1);
54                  void *g;          /* realloc sonucu ÖNCE geçici işaretçiye */
55                  /* Biri başarısız olursa kapasite ARTIRILMAZ. Önceden büyümüş
56                   * bloklar yine geçerlidir ve eski kayıtları taşır; yalnızca
57                   * fazladan yer kaplarlar (zararsız, bir sonraki denemede
58                   * realloc onları yeniden kullanır). */
59                  g = tamam ? buyut(no, yeni, sizeof *no) : NULL;
60                  if (g != NULL) { no = g; } else { tamam = false; }
61                  g = tamam ? buyut(vize, yeni, sizeof *vize) : NULL;
62                  if (g != NULL) { vize = g; } else { tamam = false; }
```

`final` ve `ort` için aynı iki satır yinelenir; ardından `tamam` yanlışsa "Bellek yetersiz" yazılır ve kayıt eklenmez, doğruysa `kapasite = yeni;` olur. **Neden geçici işaretçi?** `realloc` başarısız olursa NULL döner ve eski blok yerinde kalır; sonucu doğrudan `no`'ya atamak bloğun tek adresini silerdi (Örnek 12). Sonuç önce `g`'ye alınır ve yalnızca başarıda bağlanır (`void *` her nesne işaretçisine kendiliğinden

dönüşür). Dört büyütmeden biri başarısız olursa `kapasite` **artırılmaz**: daha önce büyümüş bloklar yine geçerlidir ve eski kayıtları taşır, yalnızca fazladan yer kaplar. **Dört paralel dizi, dört realloc** demektir: H14'te `struct` dizisiyle teke ineceğiz. `ders_istatistigi(…, &ortalama, &sapma, &enb, &enk, &basarili, &sartli)` altı sonucunu çıktı parametreleriyle verir; sütunu öbekte geçici bir diziye kopyalayıp v0.10'un 1B fonksiyonlarını ve yeni `min_max_indis`'i çağırır.

Şekil 12.11
Mini-OBS v0.12: main'in yığınındaki dört işaretçi, öbekteki dört bloğu gösterir; vize + 1 bir sonraki satıra (24 bayt) gider

```
double (*vize)[DERS_SAYISI] = malloc(kapasite * sizeof *vize);
```




```
Dolunca (her blok): g = buyut(vize, yeni, sizeof *vize); if (g != NULL) vize = g;
```

realloc sonucu önce GEÇİCİ işaretçiye; biri başarısız olursa kapasite artırılmaz, eski bloklar geçerli.

Test ve kabul. `make test`: "Tüm testler geçti (79 doğrulama: 37 kural + 14 dizi + 10 matris + 9 işaretçi + 9 bellek).". T1–T5 girdileri v0.11 ve v0.12 ile çalıştırılıp `diff` ile karşılaştırıldı (v0.11 girdisi, kapasite yanıtı atılmış aynı dosyadır): beş testte de fark yalnızca **sürüm satırı** ve **kapasite sorusu**dur, çıkış kodları aynıdır. Kapasite artışını sınayan T2 (kapasite 2, beş öğrenci):

GERÇEK DİFF (T2: KAPASİTE 2 → 4 → 8)

```
$ diff cikti_v0_11_T2_kapasite_artisi.txt cikti_v0_12_T2_kapasite_artisi.txt
2c2
<   MİNİ-OBS v0.11 · Öğrenci Not Sistemi
---
>   MİNİ-OBS v0.12 · Öğrenci Not Sistemi
4c4
<
---
> Beklenen öğrenci sayısı :
```

Bellek denetimi (gerçek). `make asan`: T1–T6 ASan + LSan altında rapor üretmedi. Valgrind her testte "All heap blocks were freed -- no leaks are possible" dedi (ör. T2 17 allocs, 17 frees, 9,432 bytes allocated); EOF ile biten T4 ve T6 da sızıntısızdır. `make bellek_siniri` ASan'ın 1 MiB ayırma sınırıyla realloc'u **gerçekten başarısız** kılar: "Büyük kapasite: 30000 -> 30000 (realloc NULL döndü; kayıtlar

korundu)". `kayitlari_birak` satırı silinmiş `sizinti_deneyi/main_sizinti.c` ise "SUMMARY: AddressSanitizer: 640 byte(s) leaked in 4 allocation(s)." raporunu verir (Valgrind: "in use at exit: 640 bytes in 4 blocks", "definitely lost: 640 bytes in 4 blocks").

Kabul ve değerlendirme. Sıfır uyarı; `make`, `make test`, `make asan`, `make valgrind` çalışır; kapasite sorusu geçersiz girdide yinelenir; realloc başarısızlığında kayıtlar korunur; 6 ile ve EOF ile çıkışta sızıntı yoktur (ayrıntı `91_Proje_MiniOBS/v0.12/mini_obs_v0_12.md`). Öğrenci çözümlerinde en sık hatalar realloc sonucunun aynı işaretçiye atanması, erken çıkışta `free` unutulması ve `sizeof vize` ile `sizeof *vize` karışıklığıdır; `make asan` ve `make valgrind` çıktısı teslime eklenmelidir. **Bilinçli sınırlar:** dört paralel dizi (H14'te `struct`), kalıcılık yok (H14'te dosya), adlar yok (H13'te karakter dizileri).

8. Alıştırmalar ve Atölye

Kodlar `kodlar/alistirmalar/` altındadır (gösterilen kesitlerin satır numaraları dosyalarla aynıdır; `#include` satırları kesitlerde gösterilmedi). Çözümler öğretim üyesindedir.

Alıştırma 12.1 ★☆☆ *Çıktı tahmini (kutu–ok izleme)*

Her satırdan sonra kutu–ok diyagramını güncelleyerek programın çıktısını tahmin edin.

alistirmalar/H12_A01_kutu_ok_izleme.c · satır 9–19

```c
int main(void)
{
    int a = 3, b = 8;
    int *p = &a;
    int *q = &b;
    *p = *q + 1;
    p = q;
    *p = *p * 2;
    printf("a = %d, b = %d, *p = %d, *q = %d\n", a, b, *p, *q);
    return 0;
}
```

İpucu: `p = q;` satırından sonra kaç ok aynı kutuya gider?

Alıştırma 12.2 ★★☆ *Çıktı tahmini (işaretçi aritmetiği)*

Dizi şeridini çizin ve p'nin her adımda hangi kutuyu gösterdiğini işaretleyin. `p[-1]` geçerli midir?

alistirmalar/H12_A02_aritmetik_tahmin.c · satır 9–20

```c
int main(void)
{
    int a[] = {4, 7, 1, 9, 2};
    int *p = a + 1;
    printf("%d ", *p);
    printf("%d ", *(p + 2));
    printf("%d ", p[-1]);
    p += 3;
    printf("%d ", *p);
    printf("%td\n", p - a);
    return 0;
}
```

Alıştırma 12.3 ★★☆ *Hata bul*

Program “en küçük 0, en büyük 0” yazar. `-Wextra` hangi uyarıyı verir? Hatayı kutu–ok diyagramıyla açıklayıp düzeltin. (`main`, `min_max(notlar, 5, &en_kucuk, &en_buyuk)` çağırıp iki değeri yazdırır.)

alistirmalar/H12_A03_hata_bul_min_max.c · satır 10–23

```c
void min_max(const int a[], int n, int *min, int *max)
{
    int enk = a[0], enb = a[0];
    for (int i = 1; i < n; i++) {
        if (a[i] < enk) {
            enk = a[i];
        }
        if (a[i] > enb) {
            enb = a[i];
        }
    }
    min = &enk;
    max = &enb;
}
```

Alıştırma 12.4 ★★☆ *Hata bul (ASan)*

Programda üç bellek hatası var; ASan’sız sürüm yine de “toplam = 55” yazar. `-fsanitize=address` ile çalıştırın, raporu okuyun ve üç hatayı düzeltin.

alistirmalar/H12_A04_hata_bul_dinamik.c · satır 11–24

```c
int main(void)
{
    int n = 5;
    int *kareler = malloc(n * sizeof *kareler);
    for (int i = 1; i <= n; i++) {
        kareler[i] = i * i;
    }
    int toplam = 0;
    for (int i = 1; i <= n; i++) {
        toplam += kareler[i];
    }
    printf("toplam = %d\n", toplam);
    return 0;
}
```

Alıştırma 12.5 ★★☆ *Parsons*

Karışık verilen satırları doğru sıraya dizip girintileyin: `dizi_toplam(bas, son)`, [bas, son) aralığındaki int’lerin toplamını bir işaretçiyle dolaşarak döndürsün. (Bu hâliyle derlenmez.)

alistirmalar/H12_A05_parsons_dizi_toplam.c · satır 7–14

```c
    return toplam;
int dizi_toplam(const int *bas, const int *son)
    for (const int *p = bas; p < son; p++) {
{
    int toplam = 0;
        toplam += *p;
}
    }
```

Alıştırma 12.6 ★★☆ *Kod tamamla*

Dört boşluğu (`___`) doldurun; beklenen çıktı "tam = 3, kesir = 0.75". (Bu hâliyle derlenmez.)

alistirmalar/H12_A06_kod_tamamla_ayir.c · satır 9–22

```c
void ayir(double x, int *tam, double *kesir)
{
    ___ = (int)x;
    ___ = x - (int)x;
}

int main(void)
{
    int t;
    double k;
    ayir(3.75, ___, ___);
    printf("tam = %d, kesir = %.2f\n", t, k);
    return 0;
}
```

Alıştırma 12.7 ★☆☆ *Kod yaz*

`void sirala3(int *a, int *b, int *c)` üç değişkeni küçükten büyüğe dizsin (Örnek 3'teki `takas`'ı kullanın). (3, 1, 2), (9, 8, 7), (1, 2, 3) ve (5, 5, 1) ile sınayın.

Alıştırma 12.8 ★★☆ *Kod yaz*

`int *dizi_kopyala(const int *a, int n)` öbekte yeni bir dizi ayırıp a'yı kopyalasın ve döndürsün; bellek yetmezse NULL. Kopyayı değiştirin ve asıl dizinin değişmediğini gösterin. Bloğu kim bırakmalı?

Alıştırma 12.9 ★★☆ *Kod yaz*

`int *filtrele(const int *a, int n, int esik, int *m)` a'daki `esik` ve üzeri elemanları yeni bir öbek dizisine koysun, eleman sayısını `*m`'ye yazsın. Önce sayıp sonra tam boyutta ayırın. Hiç eleman yoksa ne döndürmeli?

Alıştırma 12.10 ★★★ *Kod yaz*

`int ekle(int **dizi, int *n, int *kapasite, int deger)` dinamik diziye eleman eklesin, gerekirse realloc ile kapasiteyi ikiye katlasın (geçici işaretçiyle; başarısızlıkta 0 döndürüp diziyi korusun). `main`, EOF'a kadar tamsayı okuyup ters sırada yazdırsın; programı `valgrind --leak-check=full` ile doğrulayın.

Alıştırma 12.11 ★★★ *Kod yaz (meydan okuma)*

`void uygula(double *a, int n, double (*f)(double))` dizinin her elemanını f(eleman) ile değiştirsin; aynı fonksiyonu mutlak değer, kare ve yarısı için kullanın. Dizi işaretçiyle (`for (double *p = a; p < a + n; p++)`) dolaşılsın.

ALGORİTMA ATÖLYESİ · **Atölye etkinlikleri (30 dk)**

- **Kâğıt–kalem kutu–ok izleme:** Örnek 2, Alıştırma 12.1 ve 12.2'yi eşli izleyin; her satırdan sonra diyagramı yeniden çizin, sonra Python Tutor'un C görselleştiricisiyle karşılaştırın (Guo, 2013).
- **Fonksiyon üçlüsü:** `takas`, `bolum_kalan`, `dizi_min_max`'ı sıfırdan yazın; her biri için en az üç `assert` testi (normal, sınır, takma ad) ekleyin.
- **ASan ile hata avı:** `H12_13_asan_hatalari.c`'nin beş seçeneğini ASan'lı ve ASan'sız çalıştırın; Tablo 12.7'yü kendi makinenizde yeniden üretin (Linux/WSL ya da çevrim içi bir derleyici).

9. Kendini Değerlendirme

- ☐ Bir işaretçinin, gösterdiği değişkenin ve `*`/`&` ifadelerinin kutu–ok diyagramını her satırdan sonra çizebiliyorum; `int *p, q;` tuzağını ve `sizeof(int *)` = 8'i açıklayabiliyorum.
- ☐ Çıktı parametreleriyle birden çok sonuç döndürebiliyor, `scanf`'in neden `&` istediğini anlatabiliyorum.
- ☐ `p + k`'nın kaç bayt ilerlediğini hesaplayabiliyor, `a[i] ≡ *(a + i)` eşdeğerliğini ve dizi parametresinin işaretçi olduğunu açıklayabiliyorum.
- ☐ `const int *p` ile `int *const p` arasındaki farkı ve derleyicinin hangi satırı reddedeceğini söyleyebiliyorum.
- ☐ malloc/calloc/realloc/free ile dinamik dizi kurabiliyor, realloc'u geçici işaretçiyle güvenle kullanabiliyorum.
- ☐ Sızıntı, sarkan işaretçi, çifte free hatalarını tanıyıp ASan/Valgrind raporlarını okuyabiliyor; yığın–öbek ayrımını ve "Segmentation fault"un anlamını açıklayabiliyorum.

10. AP-2 Köprüsü ve Merak Kutusu

AP-2 KÖPRÜSÜ · **Bu haftanın yapı taşları AP-2'de nerede kullanılacak?**

- `takas(int *, int *)`, AP-2'deki bütün **yerinde (in-place) sıralama** algoritmalarının temel işlemidir.
- Fonksiyon işaretçileri, `qsort` ve **karşılaştırma fonksiyonu** kavramının önkoşuludur (Örnek 14'ün `uygula`'sı).
- realloc ile büyüyen dizi AP-2'de **amortize analize**; işaretçiler ise **bağlı liste, yığın (stack ADT) ve kuyruk** veri yapılarına temel oluşturur.

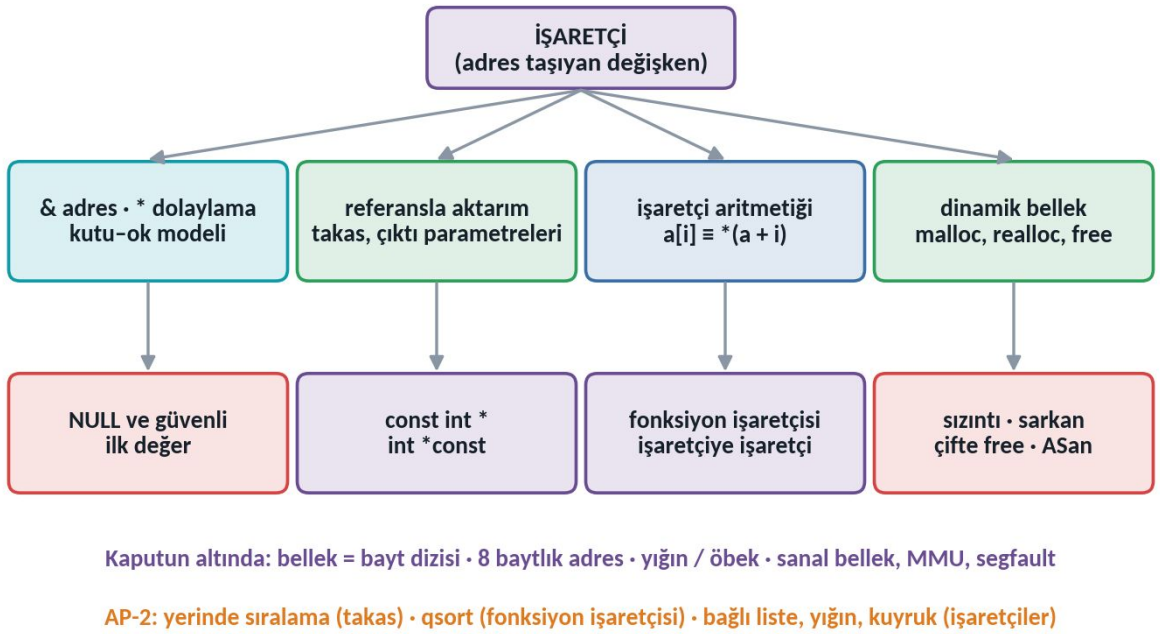
MERAK KUTUSU · **İşaretçiye işaretçi: bir işaretçiyi fonksiyonda değiştirmek**

İşaretçi de bir değişkendir; onun da adresi vardır. `int **pp = &p;` bir `int *`'ı gösterir: `*pp` p'nin kendisi, `**pp` ise x'tir. Kullanım yeri: çağıranın **işaretçisini** bir fonksiyonda değiştirmek (Alıştırma 12.10; Mini-OBS'de büyütmeyi tek bir fonksiyona taşımak da bunu gerektirirdi; bu yüzden v0.12 büyütmeyi sade biçimde `main`'de yapar). `H12_K12_isaretciye_isaretci.c`'deki `int dizi_ayir(int **hedef, int n)` bloğu ayırıp doldurur, sonra `*hedef = yeni;` ile main'deki `dizi`'yi ona yöneltir; çağrı `dizi_ayir(&dizi, 4)`'tür. Gerçek çıktı: "x = 7, *p = 7, **pp = 7", "dizi[3] = 30". Komut satırı argümanlarını tutan `char *argv[]` de bir işaretçi dizisidir (H13).

11. Özet, Kavram Haritası ve Terim Sözlüğü

- **İşaretçi** adres tutan değişkendir: `&x` adres, `*p` gösterilen nesne; x86-64'te her işaretçi 8 bayttır. Adres aktarılınca fonksiyon çağıranın değişkenine yazabilir (takas, scanf, çıktı parametreleri).
- `p + k`, k × sizeof(*p) bayt ilerler; `a[i] ≡ *(a + i)`; dizi parametresi aslında işaretçidir. `const int *p` gösterileni, `int *const p` işaretçiyi kilitler.
- Yığın otomatik, öbek elle yönetilir (`malloc`/`calloc`/`realloc`/`free`): dönüşü denetle, realloc'u geçici işaretçiye al, her bloğu bir kez bırak, sonra NULL'la.
- Sızıntı, sarkan işaretçi, çifte free çoğu zaman sessizdir: ASan ve Valgrind ile yakalanır. Segmentation fault bir koruma ihlalidir (MMU + işletim sistemi, 139).

Şekil 12.12
Hafta 12 kavram haritası



Terim sözlüğü

Tablo 12.9
Hafta 12 terimleri (Bölüm 6.8 standardıyla)

Türkçe	İngilizce	Kısa tanım
işaretçi	pointer	değeri bir adres olan değişken
adres operatörü / dolaylama	address-of operator / dereference	`&x` adres verir; `*p` gösterilen nesneye erişir
NULL işaretçi	null pointer	hiçbir nesneyi göstermeyen işaretçi değeri
referansla aktarım	pass by reference	C'de adres aktararak çağıranın değişkenini değiştirme
yığın / öbek	stack / heap	otomatik ömürlü çerçeveler / elle yönetilen bloklar
bellek sızıntısı	memory leak	bırakılmayan ve artık gösterilmeyen blok
sarkan işaretçi	dangling pointer	ömrü bitmiş nesneyi gösteren işaretçi

Türkçe	İngilizce	Kısa tanım
free sonrası kullanım / çifte free	use-after-free / double free	bırakılmış bloğa erişim / aynı bloğu iki kez bırakma
sanal bellek / sayfa	virtual memory / page	sürecin gördüğü adres uzayı / eşleme birimi (4 KiB)
bellek koruma ihlali	segmentation fault	izinsiz ya da eşlenmemiş adrese erişim (SIGSEGV)
fonksiyon işaretçisi	function pointer	bir fonksiyonun adresini tutan işaretçi

12. Kaynaklar

Bölümler için ayrıca: işaretçiler ve diziler King (2008, Bölüm 11–12), Kernighan ve Ritchie (1988, Bölüm 5) ve Deitel ve Deitel (2022, Bölüm 7); dinamik bellek ve fonksiyon işaretçileri King (2008, Bölüm 17).

Bryant, R. E., & O'Hallaron, D. R. (2016). *Computer systems: A programmer's perspective* (3rd ed.). Pearson.

Carnegie Mellon University Software Engineering Institute. (2016). *SEI CERT C coding standard: Rules for developing safe, reliable, and secure systems* (2016 ed.). Carnegie Mellon University.

Chromium Project. (t.y.). *Memory safety*. https://www.chromium.org/Home/chromium-security/memory-safety/

Deitel, P., & Deitel, H. (2022). *C how to program: With case studies introducing applications and systems programming* (9th ed., global ed.). Pearson.

Guo, P. J. (2013). Online Python Tutor: Embeddable web-based program visualization for CS education. In *Proceedings of the 44th ACM Technical Symposium on Computer Science Education* (pp. 579–584). ACM. https://doi.org/10.1145/2445196.2445368

Hoare, T. (2009, March). *Null references: The billion dollar mistake* [Conference presentation]. QCon London, London, United Kingdom. https://www.infoq.com/presentations/Null-References-The-Billion-Dollar-Mistake-Tony-Hoare/

ISO/IEC. (2018). *Information technology — Programming languages — C* (ISO/IEC Standard No. 9899:2018). International Organization for Standardization.

Kernighan, B. W., & Ritchie, D. M. (1988). *The C programming language* (2nd ed.). Prentice Hall.

King, K. N. (2008). *C programming: A modern approach* (2nd ed.). W. W. Norton.

MSRC Team. (2019, July 16). *A proactive approach to more secure code*. Microsoft Security Response Center. https://www.microsoft.com/en-us/msrc/blog/2019/07/a-proactive-approach-to-more-secure-code

Nethercote, N., & Seward, J. (2007). Valgrind: A framework for heavyweight dynamic binary instrumentation. In *Proceedings of the 28th ACM SIGPLAN Conference on Programming Language Design and Implementation* (pp. 89–100). ACM. https://doi.org/10.1145/1250734.1250746

Seacord, R. C. (2013). *Secure coding in C and C++* (2nd ed.). Addison-Wesley.

Serebryany, K., Bruening, D., Potapenko, A., & Vyukov, D. (2012). AddressSanitizer: A fast address sanity checker. In *2012 USENIX Annual Technical Conference (USENIX ATC 12)* (pp. 309–318). USENIX Association.